

MIIA: Visión Artificial, PEC 1

Alejandro de Cora

2026-11-11

Tabla de contenidos

MIIA: Visión Artificial, PEC 1 **3**

1 Eliminación de ruido **4**
1.1 Ejercicio 1.1 . . . 4
1.2 Ejercicio 1.2 . . . 4

2 Transformaciones geométricas **7**
2.1 Ejercicio 2.1 . . . 7
2.2 Ejercicio 2.2 . . . 9
2.2.1 Implementación de las transformaciones de forma individual . . . 9
2.2.2 Implementación de las transformaciones compuestas . . . 9
2.2.3 Transformar la región de interés (ROI) . . . 12
2.3 Ejercicio 2.3 . . . 14
2.3.1 Deformación de una imagen . . . 14
2.3.2 Transformación no lineal . . . 15

3 Calibración de la cámara **18**
3.1 Ejercicio 3.1 . . . 18
3.1.1 Evaluación del error de calibración . . . 20
3.2 Ejercicio 3.2 . . . 20

4 Flujo óptico y seguimiento **24**
4.1 Ejercicio 4.1 . . . 24
4.1.1 Lucas-kanade optical flow . . . 24
4.1.2 Dense optical flow . . . 27
4.2 Ejercicio 4.2 . . . 28

5 Segmentación sin conocimiento del dominio **32**
5.1 Umbralización . . . 32
5.1.1 Umbralización global . . . 32
5.1.2 Umbralización local . . . 32
5.2 K-means clustering . . . 35
5.3 Segmetación con MeanShift . . . 38
5.4 Segmentación Watershed . . . 40

Referencias **44**

MIIA: Visión Artificial, PEC 1

Prueba de evaluación continua 1.

El material fuente está escrito principalmente como notebooks y se renderiza con Quarto [1].

1 Eliminación de ruido

1.1. Ejercicio 1.1

La aplicación **image-playground** para realizar procesamiento de imágenes se encuentra desplegada en huggingface [2]: https://huggingface.co/spaces/vichel0creg0/image-playground.

También puede ejecutarse de forma local utilizando el comando:

```
# Lanza la interfaz UI de la aplicación image-playground de forma local
!uv run miia-pec1 ui --port 5432
```

1.2. Ejercicio 1.2

Para reducir el ruido, se ha diseñado un pipeline de cuatro etapas.

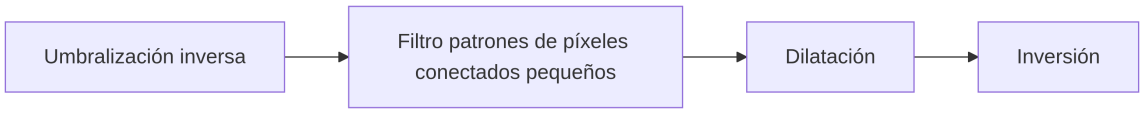
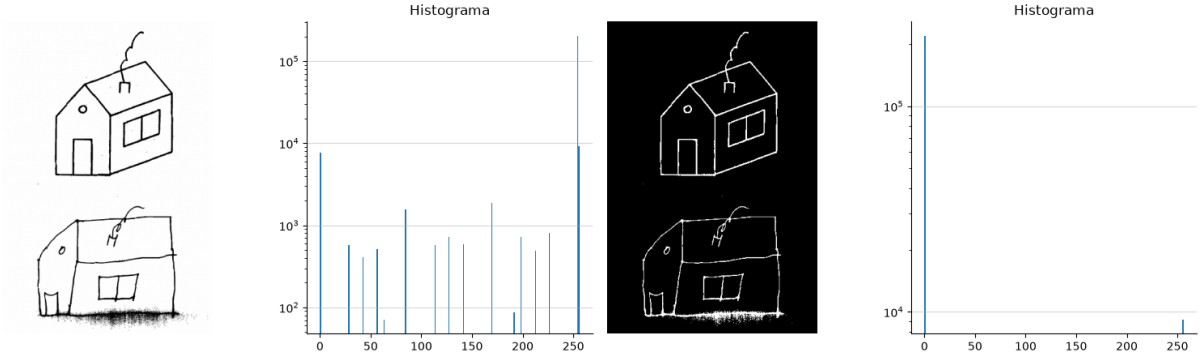


Figura 1.1: Pipeline de eliminación de ruido

En la Figura 1.2 se muestra la primera etapa del pipeline, consiste en una umbralización binaria inversa. Esta operación permite eliminar tonalidades de grin intermedias y obtener una representación binaria en la que los trazos de los dibujos aparecen en blanco y el fondo en negro.

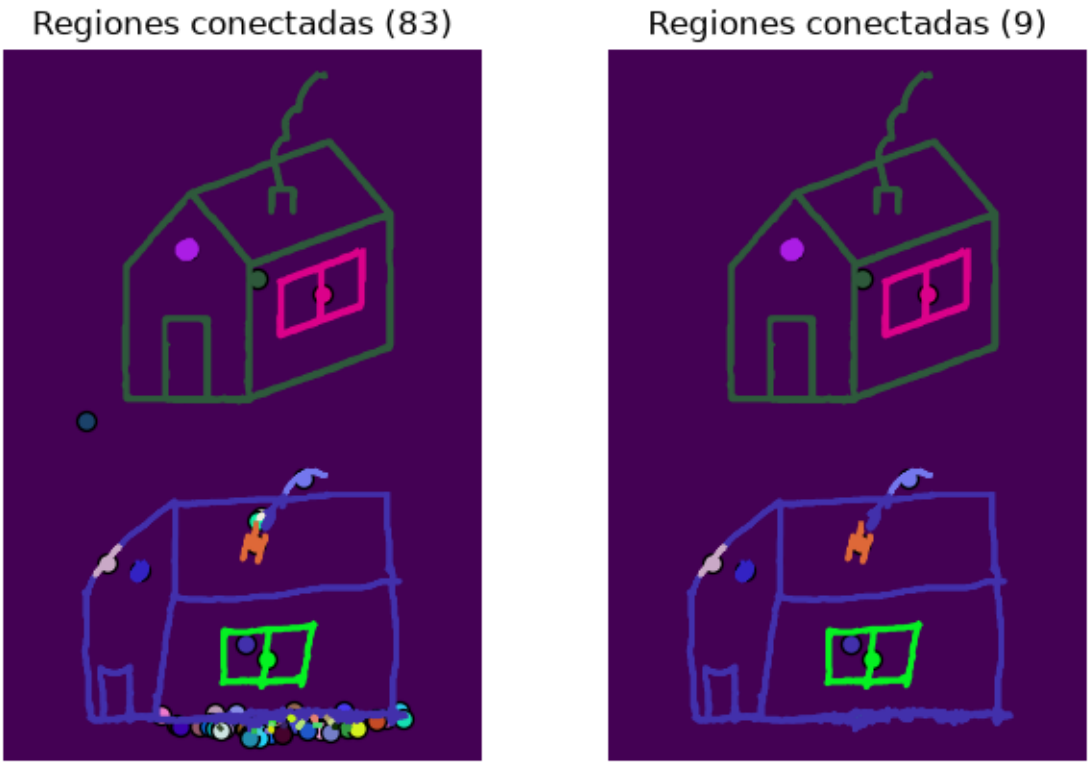


(a) Imagen original y su histograma (b) Imagen binarizada y su histograma

Figura 1.2: Primer paso del pipeline: Umbralización inversa

Para loa segunda etapa del pipeline se utiliza el procedimiento propuesto por Moracho [3], que se basa en el filtrado de los componentes conectados según su área.

En la Figura 1.3 se muestra como el número de componentes conectados se reduce de 83 a 9, eliminando gran parte de las pequeñas regiones asociadas al ruido.



(a) Antes del filtrado

(b) Después del filtrado

Figura 1.3: Componentes conectadas

Finalmente, Figura 1.4, se aplica una dilatación morfológica para aumentar ligeramente el grosor de la los trazos y reforzar las líneas que hayan quedado demasiado finas tras las etapas anteriores.

En la Figura 1.5, se presenta una comparación de las imágenes de entrada con los resultados obtenidos en cada etapa de pipeline.

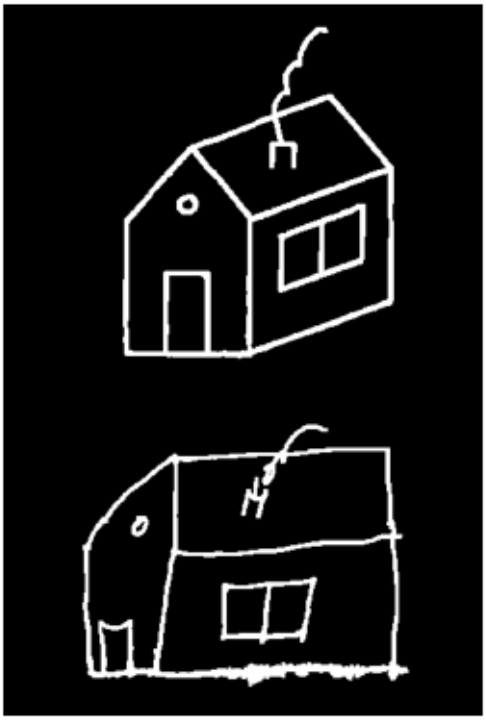


Figura 1.4: Dilatación morfológica, engrosamiento de los trazos de la imagen

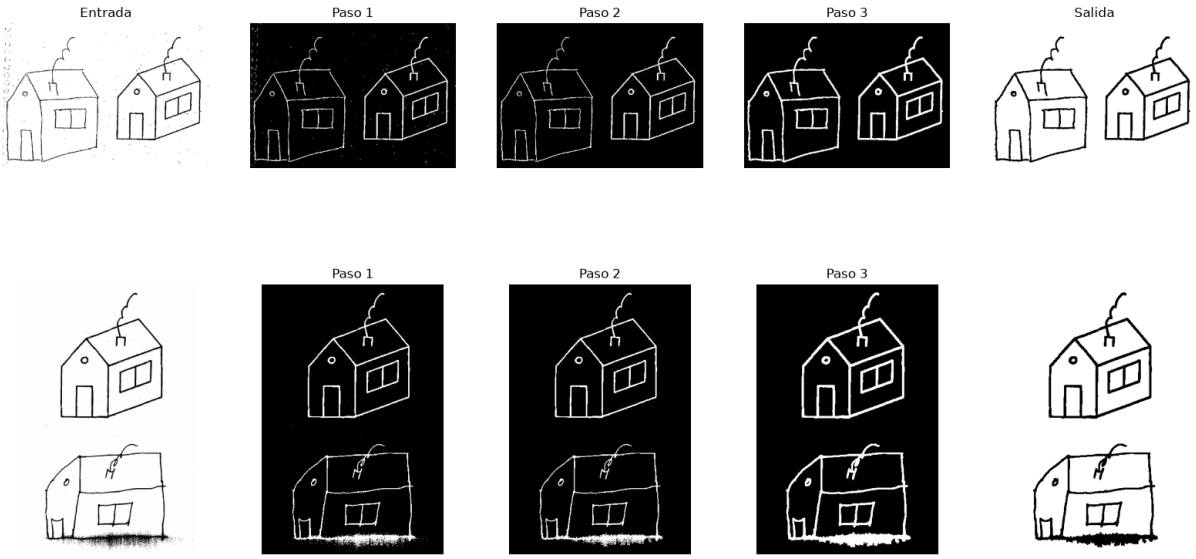


Figura 1.5: Estapas del pipeline de filtrado de ruído

2 Transformaciones geométricas

2.1. Ejercicio 2.1

Se puede rotar una imagen respecto al centro de la imagen, aplicando una transformación afín utilizando la función `cv2.getRotationMatrix2D()` [4].

La matriz de rotación de OpenCV [4] incorpora el centro de rotación y un factor de escala.

$$M = \begin{bmatrix} \alpha & \beta & (1-\alpha)c_x - \beta c_y \\ -\beta & \alpha & \beta c_x + (1-\alpha)c_y \end{bmatrix}$$

donde

$$\alpha = s\cos\theta$$
$$\beta = s\sin\theta$$

Como se puede ver en Listado 2.1 esta matriz es equivalente a realizar las siguientes transformaciones de forma encadenada:



Figura 2.1: Rotar una imagen respecto a su centro

```
np.allclose(M[:2,:], rot) = True
```

Al rotar una imagen las dimensiones del marco de salida cambian, si no ajustamos las dimensiones al nuevo marco perdemos información de la imagen original.

Para evitar la pérdida de información durante la rotación de una imagen, es necesario ajustar las dimensiones del marco de salida de modo que contenga completamente la imagen transformada [5].

Las nuevas dimensiones pueden calcularse a partir de las proyecciones horizontales y verticales de los lados de la imagen original, como se ilustra en Figura 2.3.

Una vez calculadas estas dimensiones, es necesario situar el centro de la imagen original en el centro del nuevo marco, evitando así que la imagen rotada quede recortada.



(a) Imagen original (b) Imagen rotada 45º en el frame orginal

Figura 2.2: Rotación

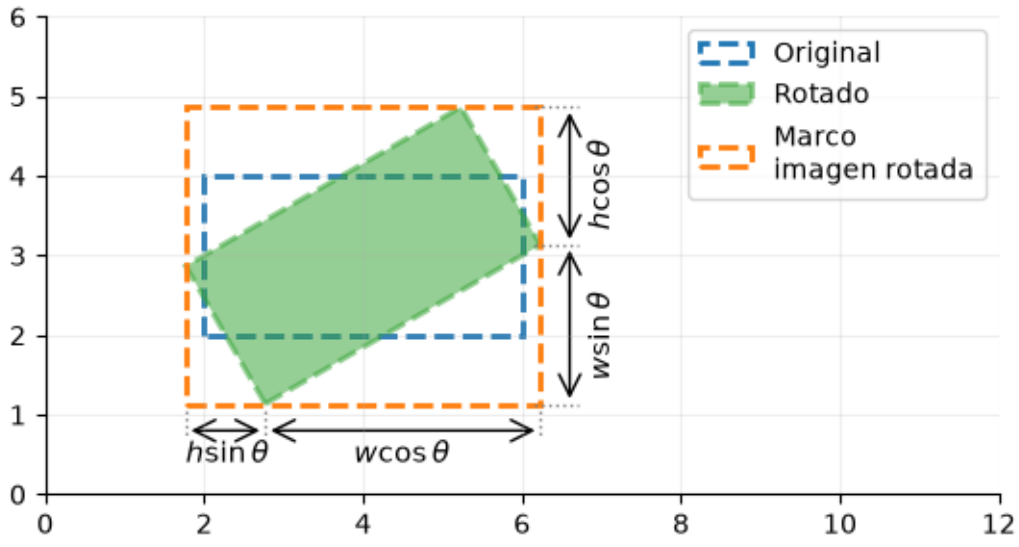


Figura 2.3: Dimensiones proyectadas

Listado 2.1 Composición de transformaciones afines

```
# Escogemos el centro de rotación
c_h, c_w = 100, 200

# Matriz de rotación con OpenCV
rot = cv.getRotationMatrix2D((c_w, c_h), 30, 1)

# Rotación respecto (c_w, c_h) con transformaciones afines

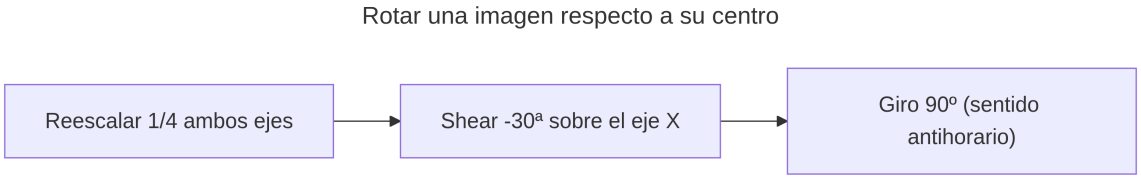
# Trasladar el centro de la imagen al origen de coordenadas
T1 = mtm.TRANSLATE(t_x=-c_w, t_y=-c_h)
# Rotar la imagen respecto al origen de coordeandas
R = mtm.ROTATE(30)
# Trasladar la imagen de vuelta al su posición original
T2 = mtm.TRANSLATE(t_x=c_w, t_y=c_h)

# Matriz de rotación calculada
M = T2 @ R @ T1

print(f'np.allclose(M[:2,:], rot) = {np.allclose(M[:2,:], rot)}')
```

2.2. Ejercicio 2.2

2.2.1. Implementación de las transformaciones de forma individual



2.2.2. Implementación de las transformaciones compuestas

En la composición de transformaciones geométricas, el orden de las transformaciones afecta al resultado, ya que la multiplicación de matrices no es conmutativa [6]:

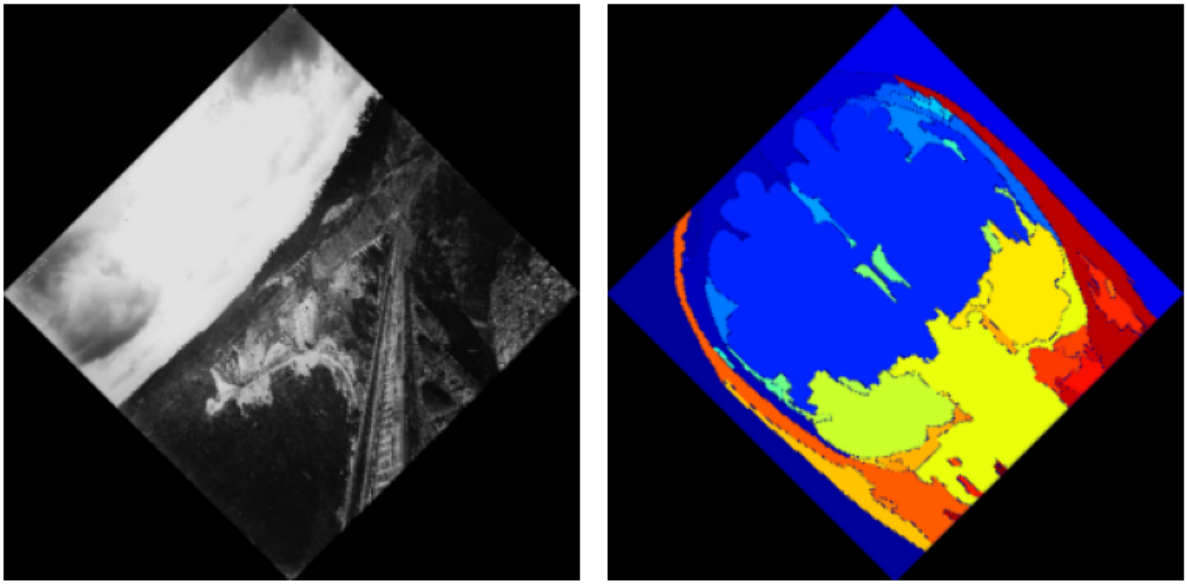
$$\mathbf{AB} \neq \mathbf{BA}$$

Por ejemplo, dadas las matrices:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$$

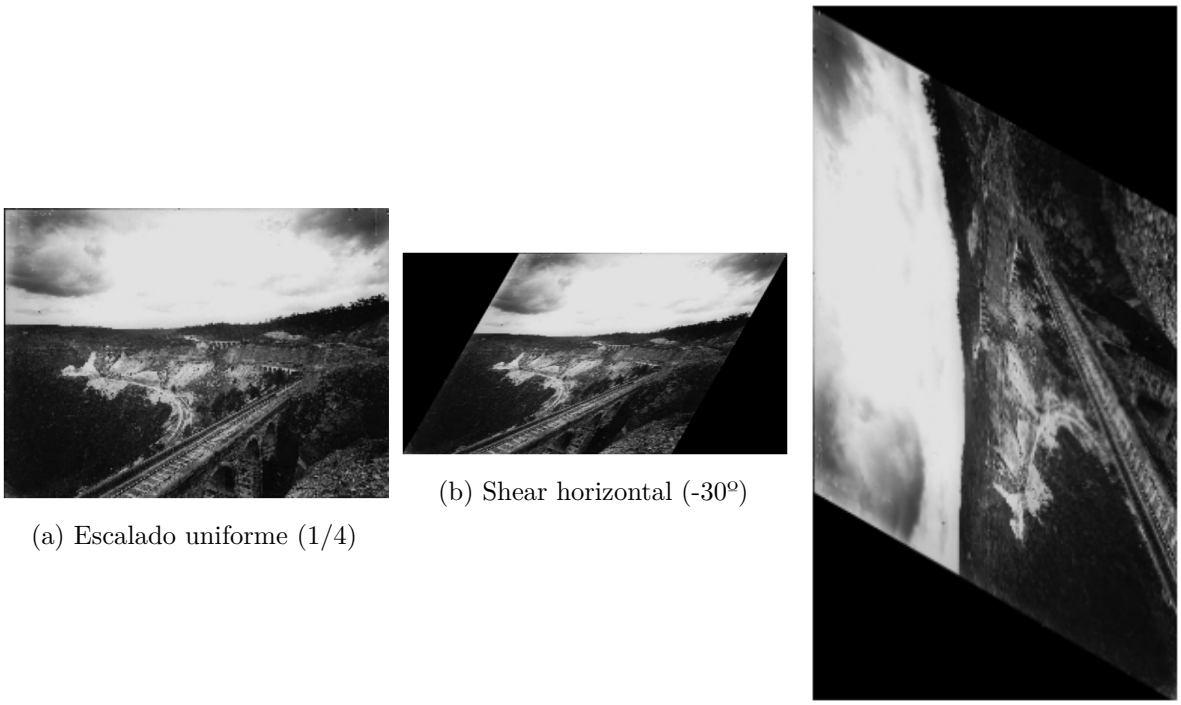
Se obtienen los siguientes productos:

$$\mathbf{AB} = \begin{bmatrix} 0 & 1 \\ 0 & 3 \end{bmatrix}, \qquad \mathbf{BA} = \begin{bmatrix} 3 & 4 \\ 0 & 0 \end{bmatrix}$$



(a) Imagen zigzag (b) Imagen brainLabels

Figura 2.4: Escalado y rotación de imágenes con ajuste del marco de salida



(a) Escalado uniforme (1/4)

(b) Shear horizontal (-30º)

(c) Rotación antihoraria (90ª)

Figura 2.5: Composición de transformaciones afines sobre una imagen

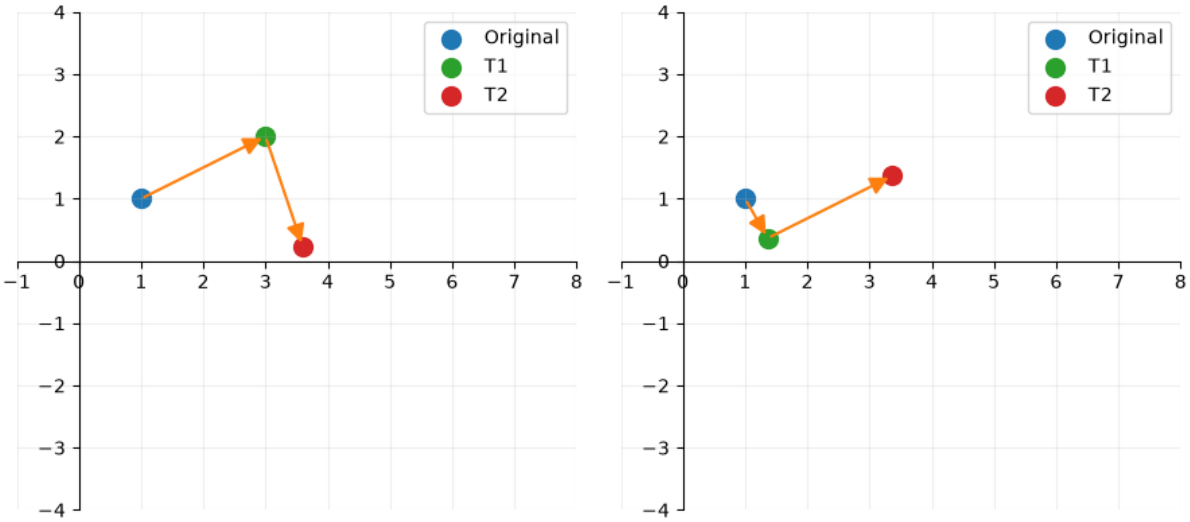
Sin embargo, la multiplicación de matrices sí cumple la propiedad asociativa, que permite agrupar varias transformaciones en una única matriz sin alterar el resultado.

Para un punto representado mediante el vector columna $\mathbf{x}$, si primero se aplica la transformación $\mathbf{T}_1$ y posteriormente $\mathbf{T}_2$, se obtiene:

$$\mathbf{x}' = \mathbf{T}_2(\mathbf{T}_1\mathbf{x}) = (\mathbf{T}_2\mathbf{T}_1)\mathbf{x} = \mathbf{T}_C\mathbf{x}$$

donde $\mathbf{T}_C = \mathbf{T}_2\mathbf{T}_1$ representa la matriz de transformación compuesta.

Como se muestra en Figura 2.6, el orden de aplicación de las transformaciones afecta al resultado final. En el primer caso se aplica una traslación seguida de una rotación, mientras que en el segundo se invierte el orden, obteniéndose posiciones finales diferentes para un mismo punto inicial.



(a) Traslación seguida de rotación

(b) Rotación seguida de traslación

Figura 2.6: Efecto del orden de aplicación de las transformaciones geométricas

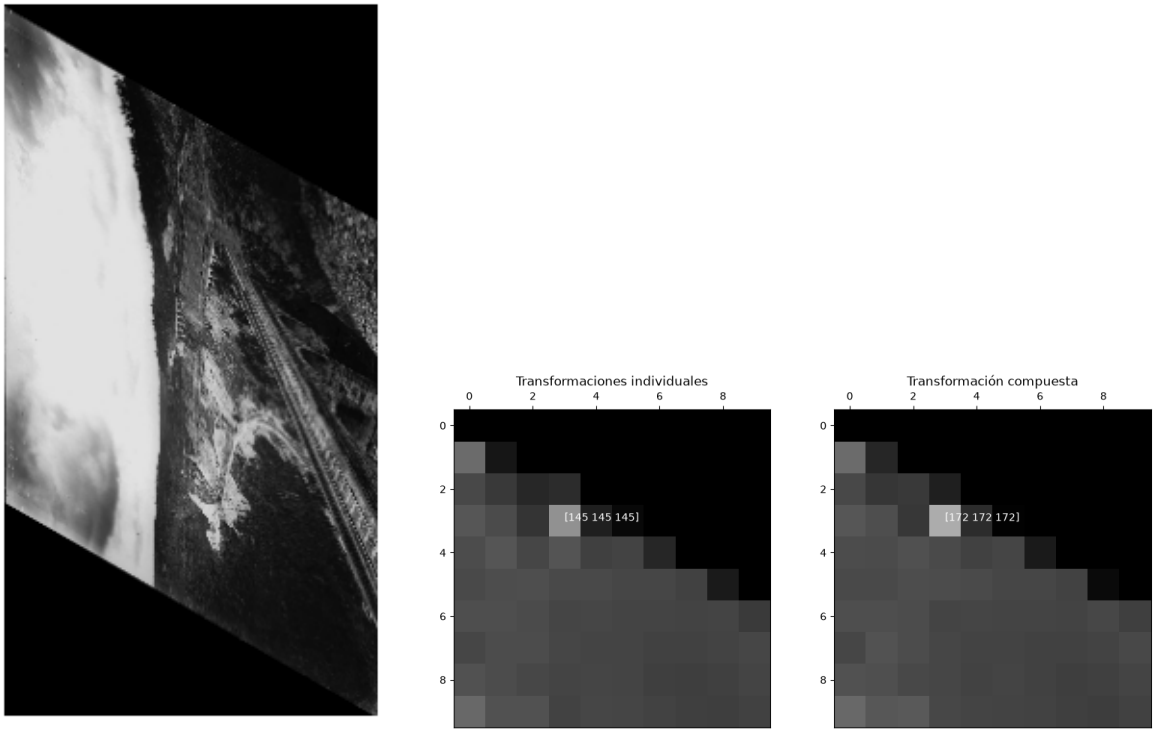
El resultado de realizar las transformaciones mediante composición de matrices, se muestra en la Figura 2.7a, geométricamente el resultado es el mismo. Sin embargo, si observamos el detalle de las intensidades de los píxeles resultantes, en la figura Figura 2.7b, no son iguales. Cada transformación sobre la imagen realiza una interpolación, en las transformaciones individuales por tanto se están aplicando tres interpolaciones, por una en el caso de la transformación compuesta.

Al realizar transformaciones geométricas de 2D sobre una imagen, las coordenadas de los píxeles de la imagen resultante no tienen por qué coincidir con las de la imagen original.

Para determinar el valor de cada píxel de la imagen de destino, OpenCV utiliza un proceso de **mapeo inverso**, que consiste en recorrer los píxeles de la salida y calcular sus coordenadas correspondientes en la imagen de origen [7].

$$\text{dst}(x, y) = \text{src}\left(f_x(x, y), f_y(x, y)\right)$$

donde $f_x(x, y)$ y $f_y(x, y)$ representan las coordenadas de la imagen original correspondientes al píxel (x, y) de la imagen destino.



(a) Resultado de la transformación (b) Detalle de las imágenes transformadas

Figura 2.7: Aplicación de transformaciones afines mediante composición de matrices

Estas coordenadas toman valores decimales y, por tanto, no tienen porque tener una correspondencia exacta con un la posición de un píxel de la imagen original. Para estimar la intensidad del píxel de la imagen de salida se aplican diferentes métodos de interpolación.

La interpolación por el vecino más cercano consiste en asignar a cada píxel de destino el valor del píxel de origen cuya posición sea la más cercana a las coordenada calculadas [8]. Como se muestra en Figura 2.8, el valor del punto interpolado se obtiene directamente del píxel más cercano.

Este método de interpolación es computacionalmente poco costoso, no tiene que realizar promedios entre píxeles cercanos, y especialmente útil cuando es importante conservar los valores orginales de los píxeles, por ejemplo si vamos a aplicar máscaras de segmentación es pasos posteriores.

2.2.3. Transformar la región de interés (ROI)

Para transformar la **región de interés (ROI)** hasta la posición objetivo, primero es necesario determinar las coordenadas de sus vértices en la imagen original.

Como se muestra en Figura 2.9, estas coordenadas pueden calcularse a partir de las dimensiones de la región, su orientación y las proyecciones de sus lados sobre los ejes de coordenadas.

Tras determinar las coordenadas de los vertices de la región de interés, se propone una secuencia de transformaciones geométricas para situarla en la posición objetivo.

Como se ilustra en Figura 2.10, el procedimiento consta de tres operaciones: primero se aplica una rotación de 30º en sentido antihorario para alinear la región con los ejes de coordenadas, luego se realiza una traslación para situar el vertice superior izquierdo en el origen y finalmente se aplica un escalado uniforme para reducir sus dimensiones a la mitad.

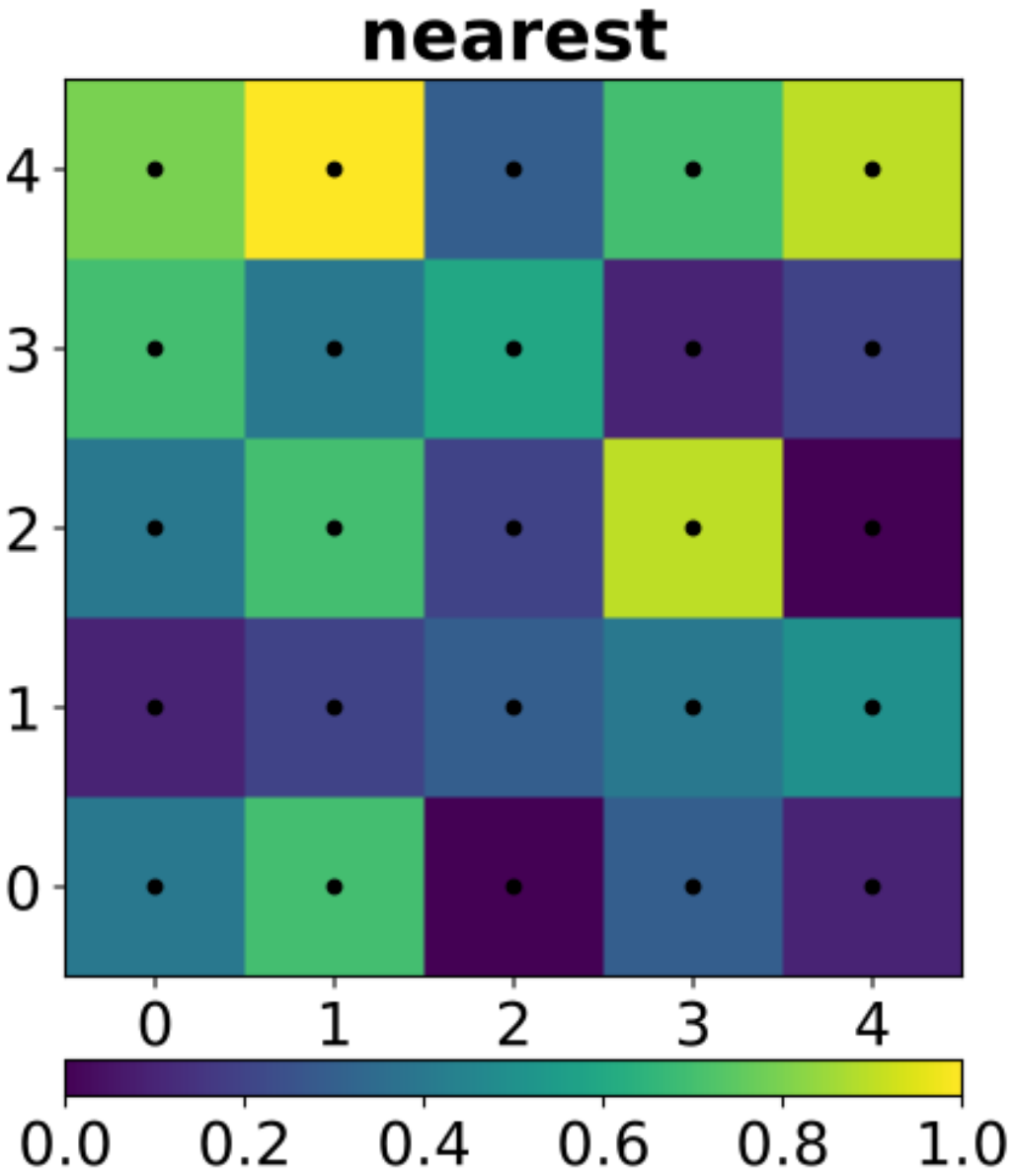


Figura 2.8: Interpolación 2D del vecino más cercano

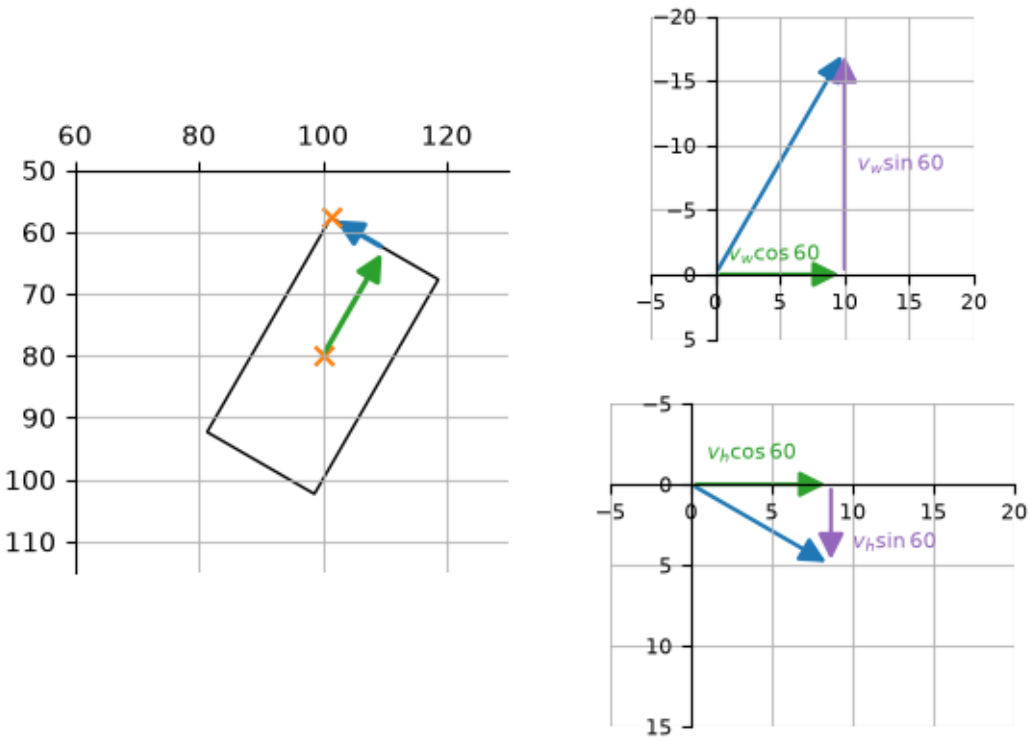
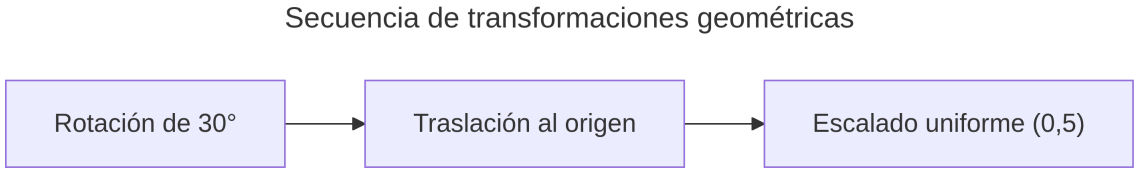


Figura 2.9: Determinación de las coordenadas de los vértices de la región de interes



2.3. Ejercicio 2.3

2.3.1. Deformación de una imagen

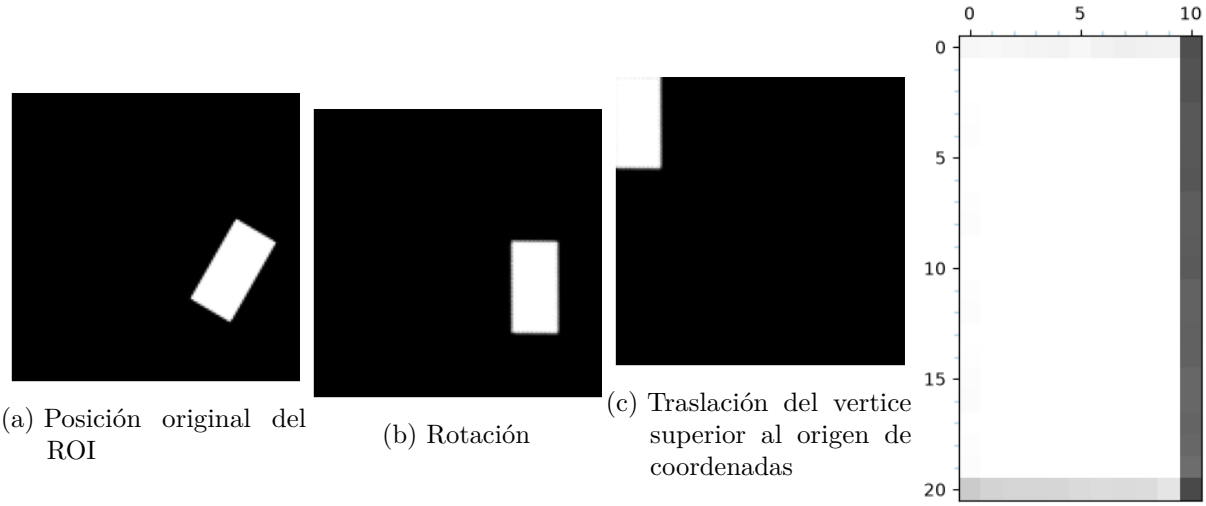
La deformación de la imagen se realiza mediante una transformación lineal por tramos, en la que se aplica un factor de escala horizontal diferente a cada mitad de la imagen, tal como se define en Ecuación 2.1.

$$x' = \begin{cases} \frac{2}{3}x, & x \leq W/2 \\ \frac{W}{3} + \frac{4}{3}(x - W/2), & x > W/2 \end{cases} \tag{2.1}$$

$$y' = y$$

Como se observa en Figura 2.11, la transformación modifica únicamente las coordenadas horizontales, manteniendo las verticales sin cambios:

- Mitad izquierda: se aplica un factor de escala de $2/3$, comprimiendo su anchura original de $W/2$ a $W/3$.



(a) Posición original del ROI

(b) Rotación

(c) Traslación del vertice superior al origen de coordenadas

(d) Escalado de la imagen

Figura 2.10: Secuencia de transformaciones para recolocar la región de interés

- Mitad derecha: se aplica un factor de escala de $4/3$, expandiendo su anchura original de $W/2$ a $2W/3$.

Finalmente, ambas regiones se combinan para obtener la imagen transformada, conservando la anchura total original W y manteniendo la continuidad en el punto de unión entre las dos mitades.

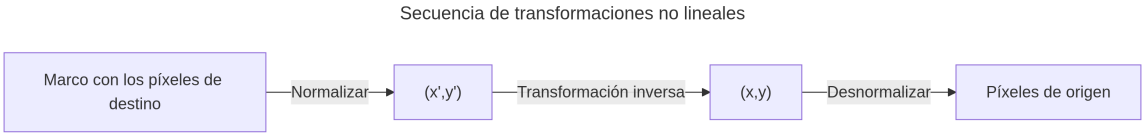
2.3.2. Transformación no lineal

Se considera la siguiente transformación geométrica:

$$\begin{aligned} y' &= y + 0.4(x - 0.5)^2 - 0.1 \\ x' &= 2x \end{aligned} \tag{2.2}$$

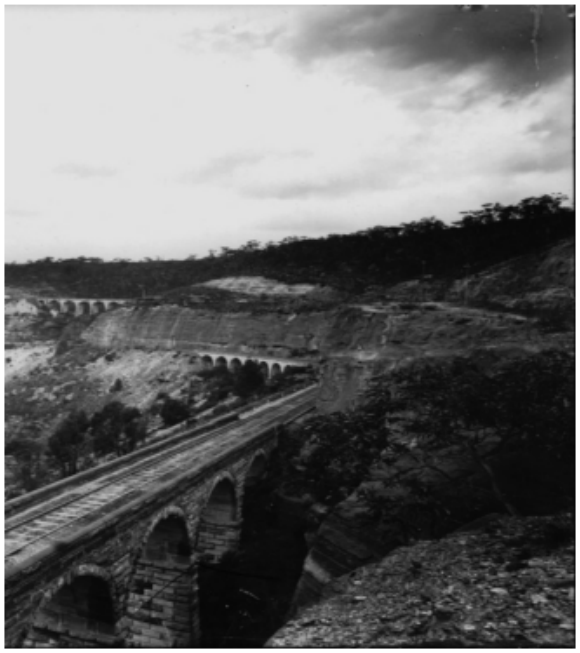
La transformación Ecuación 2.2 es **no-lineal**, debido a la dependecia cuadrada de la coordenada x, a diferencia de las trasnformaciones afines esta transformación no conserva las líneas rectas, como se observa en la figura Figura 2.12.

Aplicamos la tranformación Ecuación 2.2 a la imagen siguiendo la secuencia:

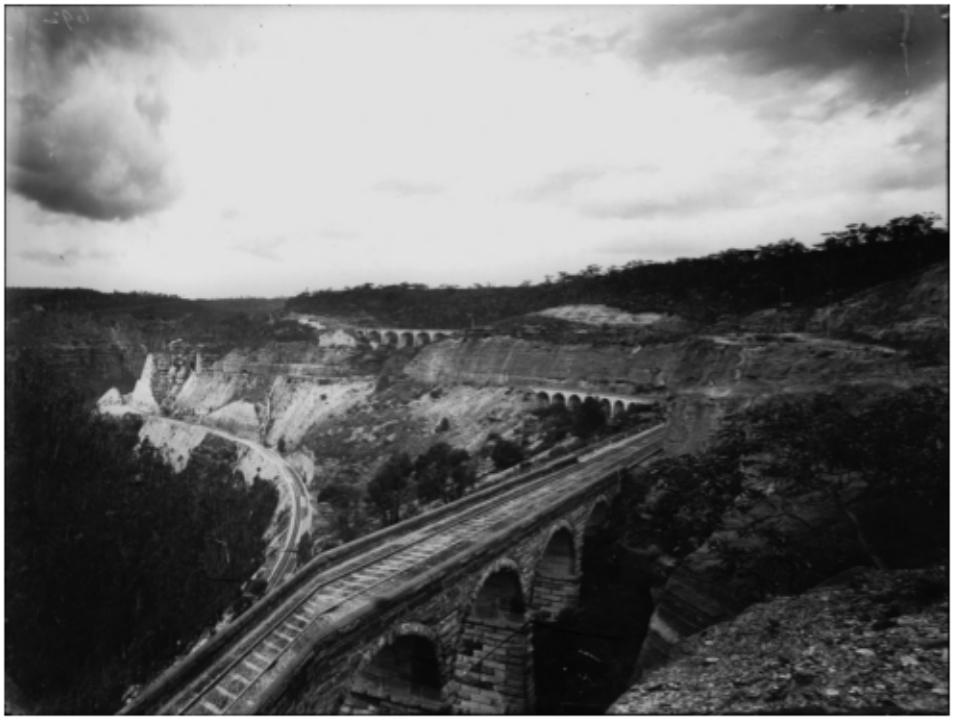




(a) Mitad izquierda: escala 2/3



(b) Mitad derecha: escala 4/3



(c) Imagen resultante

Figura 2.11: Deformación horizontal mediante una transformación lineal por tramos

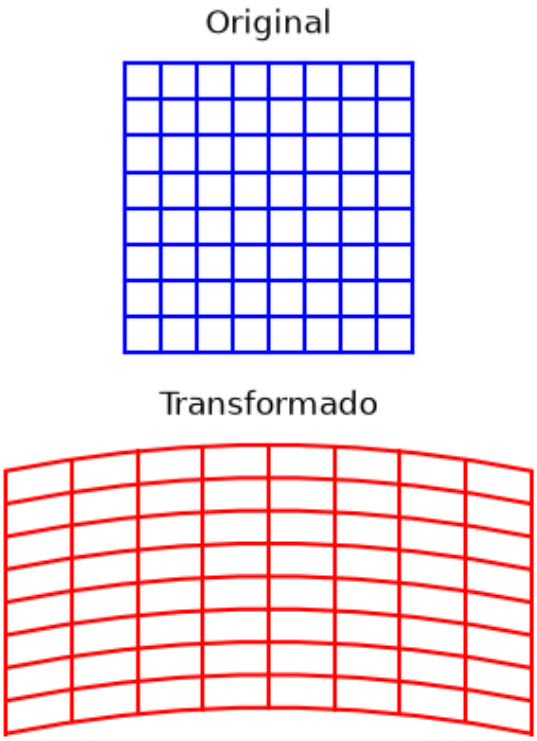


Figura 2.12: Transformación no lineal

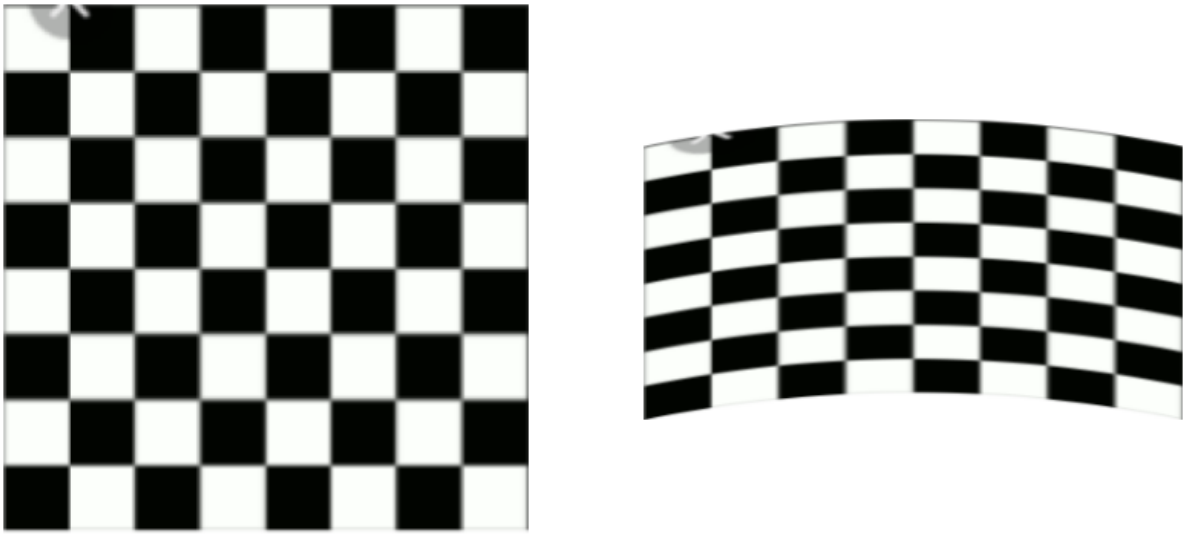


Figura 2.13: Transformación no lineal del tablero de ajedrez

3 Calibración de la cámara

3.1. Ejercicio 3.1

Para realizar la calibración de la cámara, he utilizado un patrón de tablero de ajedrez obtenido de las plantillas que ofrece OpenCV. He impreso el patrón y lo he fijado a una superficie rígida para minimizar posibles deformaciones del papel que pudiesen afectar a la calibración de la cámara.

Para la captura de la imágenes he implementado un pequeño programa CLI que puede ejecutar con el comando `uv run miia-pec1 capture`, que permite visualizar en tiempo real las esquinas detectadas para facilitar la captura del tablero en diferentes posiciones y orientaciones generando imágenes válidas para la calibración.

En Figura 3.1 se muestran las imágenes seleccionadas para el proceso de calibración.

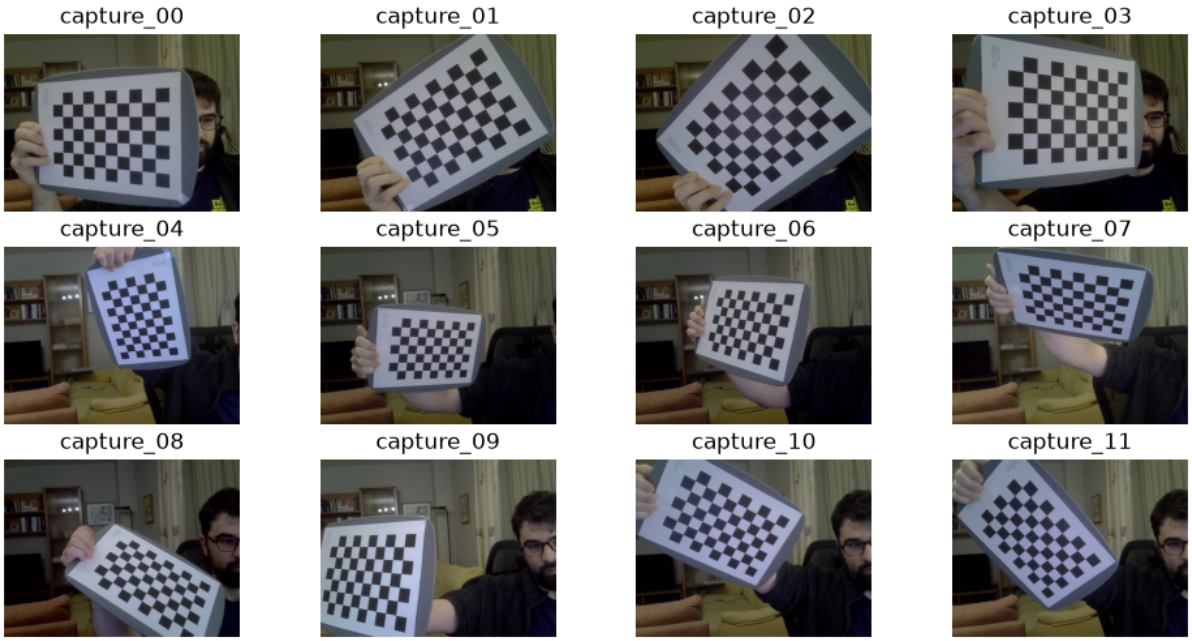


Figura 3.1: Imágenes del patrón de ajedrez utilizadas para la calibración

Posteriormente, el comando `uv run miia-pec calibrate` realiza la calibración de la cámara a partir de las corresponcias entre los puntos del patrón y sus proyecciones en las imágenes, estima los parámetros de la cámara y los coeficientes de distorsión.

Como se observa en Figura 3.2, el algoritmo identifica las esquinas interiores del tablero de ajedrez, que se utilizan como puntos de referencia para calcular los parámetros de calibración.

En Tabla 3.1 se muestran los parámetros obtenidos durante la calibración, la matriz intrínseca $\mathbf{K}$ y los coeficientes de distorsión $\mathbf{D}$.

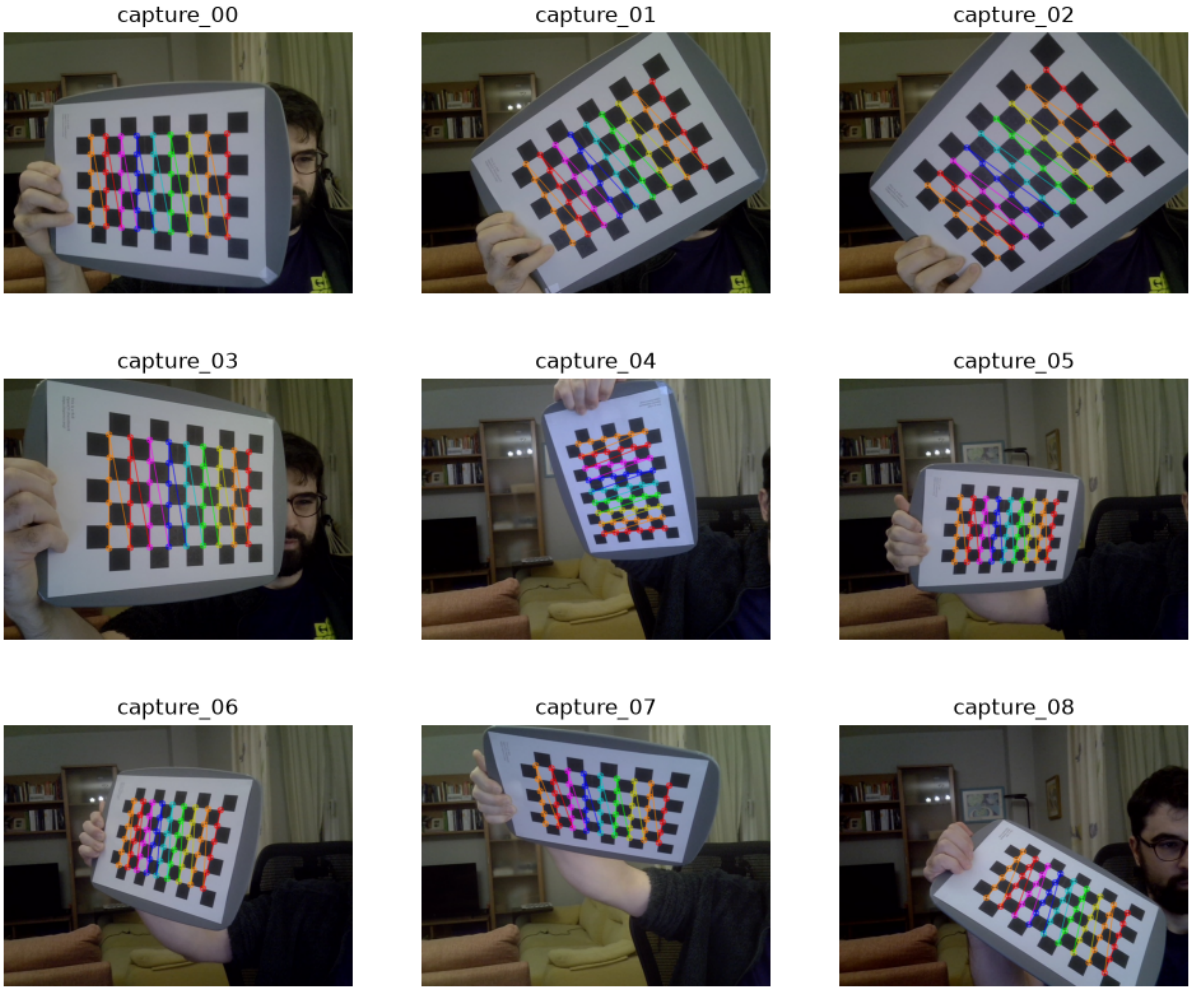


Figura 3.2: Detección de esquinas interiores del patrón de calibración

La matriz intrínseca **K** recoge los parámetros que describen la proyección de la cámara. En particular, las distancias focales expresadas en píxeles son $f_x \approx 534.16$ y $f_y \approx 534.25$, mientras que el punto principal se encuentra en $(c_x, c_y) \approx (341.71, 232.05)$.

El vector de distorsión **D** contiene los coeficientes utilizados para modelar las distorsiones radiales y tangenciales de la lente. En el modelo habitual de cinco coeficientes de OpenCV, estos se ordenan como $(k_1, k_2, p_1, p_2, k_3)$.

Estos parámetros permiten corregir la distorsión de las imágenes y realizar proyecciones geométricas más precisas.

Tabla 3.1: Resultados de la calibración de la cámara

Parámetro	Valor
Matriz intrínseca K	$\begin{bmatrix} 701.7948 & 0.0000 & 316.8115 \\ 0.0000 & 702.0579 & 236.2497 \\ 0.0000 & 0.0000 & 1.0000 \end{bmatrix}$
Distorsión D	$(0.0183, 0.6575, -0.0077, 0.0025, -1.8116)$

3.1.1. Evaluación del error de calibración

Tras completar la calibración, se analiza el error de reproyección, que mide la diferencia entres las posiciones de las esquinas detectadas y las estimadas utilizando los parámetros de calibración obtenidos.

En Figura 3.3 se puede observar el error de calibración para cada imagen, junto con el error de calibración global RMS.

Los resultados muestran errores relativamente reducidos, con pequeñas diferencias entre las imágenes utilizadas. El error RMS global es aproximadamente de 0,276 píxeles, lo que indica una buena correspondencia entre las observaciones y el model estimado.

> **Nota**
>
> Un RMS bajo no garantiza una calibración perfecta, es importante que las imágenes cubran diferentes orientaciones y regiones del sensor.

En Figura 3.4 se compara la imagen original con su versión corregida. Aunque las diferencias son sutiles, se aprecia una ligera rectificación de las líneas y los bordes, especialmente en las zonas periféricas de la imagen.

3.2. Ejercicio 3.2

¿Como podemos estimar la pose de una un objeto respecto a la cámara?

Partimos de unos puntos 3D conocidos del objeto y detectamos sus correspondientes posiciones 2D en la imagen.

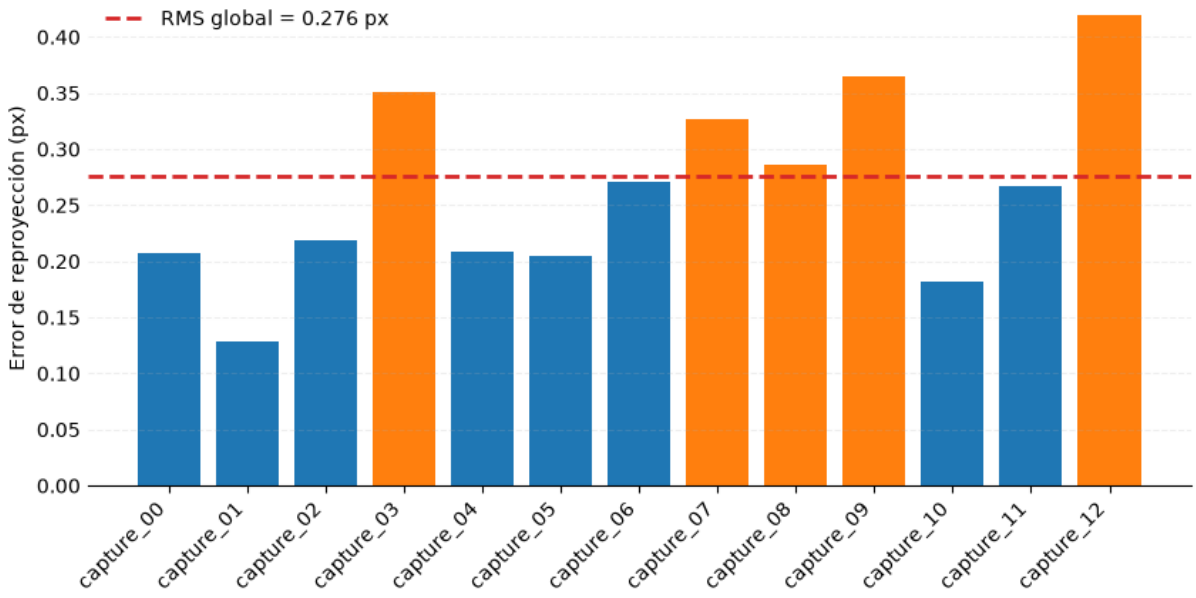


Figura 3.3: Error de reproyección por imagen y RMS global



(a) Imagen original

(b) Imagen corregida

Figura 3.4: Comparación de la imagen original y la imagen corregida mediante calibración

Tabla 3.2: Correspondencias entre los puntos 3D del tablero y sus posiciones 2D detectadas en la imagen

Puntos 3D conocidos	Posiciones 2D en la imagen
$(0.0000, 0.0000, 0.0000)$	$(380.1252, 332.5765)$
$(1.0000, 0.0000, 0.0000)$	$(360.6099, 333.1312)$
$(2.0000, 0.0000, 0.0000)$	$(340.7096, 333.4769)$
$(3.0000, 0.0000, 0.0000)$	$(320.2885, 333.8613)$
$(4.0000, 0.0000, 0.0000)$	$(299.3304, 334.4000)$

A partir de las correspondencias entre los puntos 3D y 2D, la matriz intrínseca y los coeficientes de distorsión de la cámara, `cv.solvePnP()` permite estimar la rotación y la traslación del objeto respecto a la cámara. Estos parámetros se calculan buscando que las proyecciones de los puntos 3D se ajusten lo mejor posible a los puntos 2D detectados. Esta transformación se ilustra en la Figura 3.5.

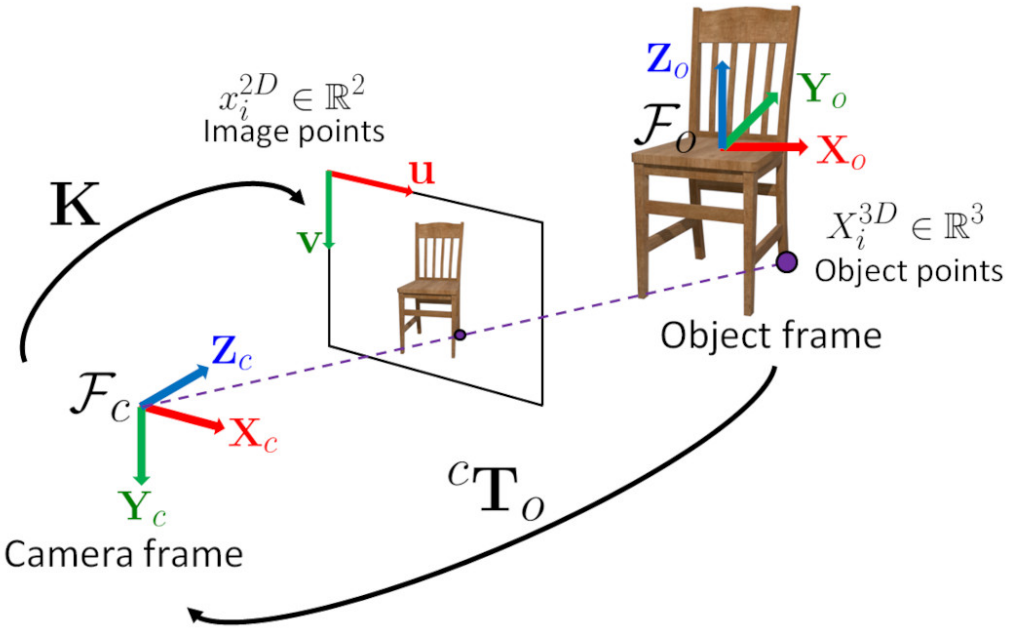
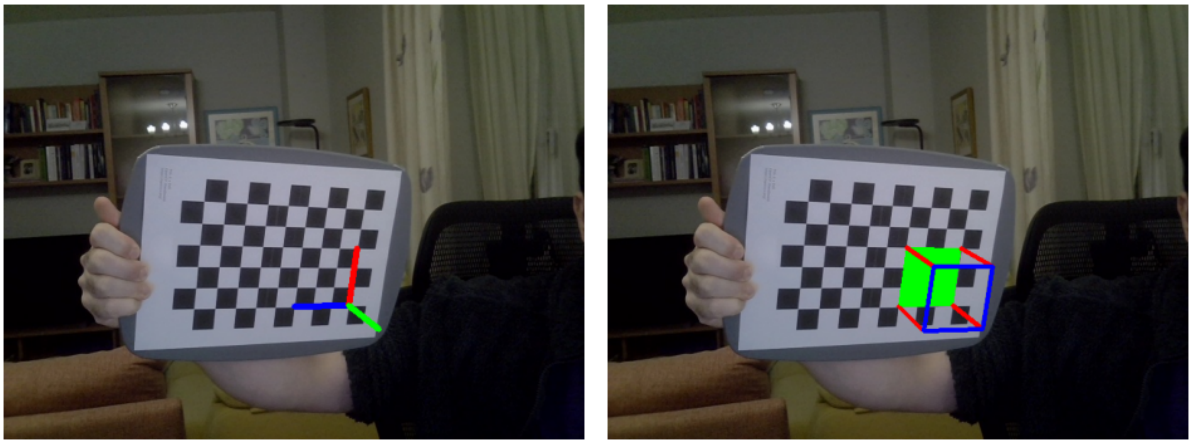


Figura 3.5: Sistemas de coordenadas de la cámara y del objeto y proyección de puntos 3D en el plano de imagen. Fuente: OpenCV [9].



(a) Proyección de los ejes de coordenadas del tablero (b) Proyección de un cubo virtual sobre el tablero

Figura 3.6: Estimación de la pos del tablero y visualización mediante elmentos virtuales 3D

Este procedimiento podría aplicarse a la cabeza de una persona, sería necesario un modelo facial 3D de referencia junto con el sistema capaz de detectar las posiciones 2D de puntos significativos. A partir de las correspondencias, `cv.solvePnP()` permitiría estimar la orientación de la cabeza respecto a la cámara, como resume la Figura 3.7.

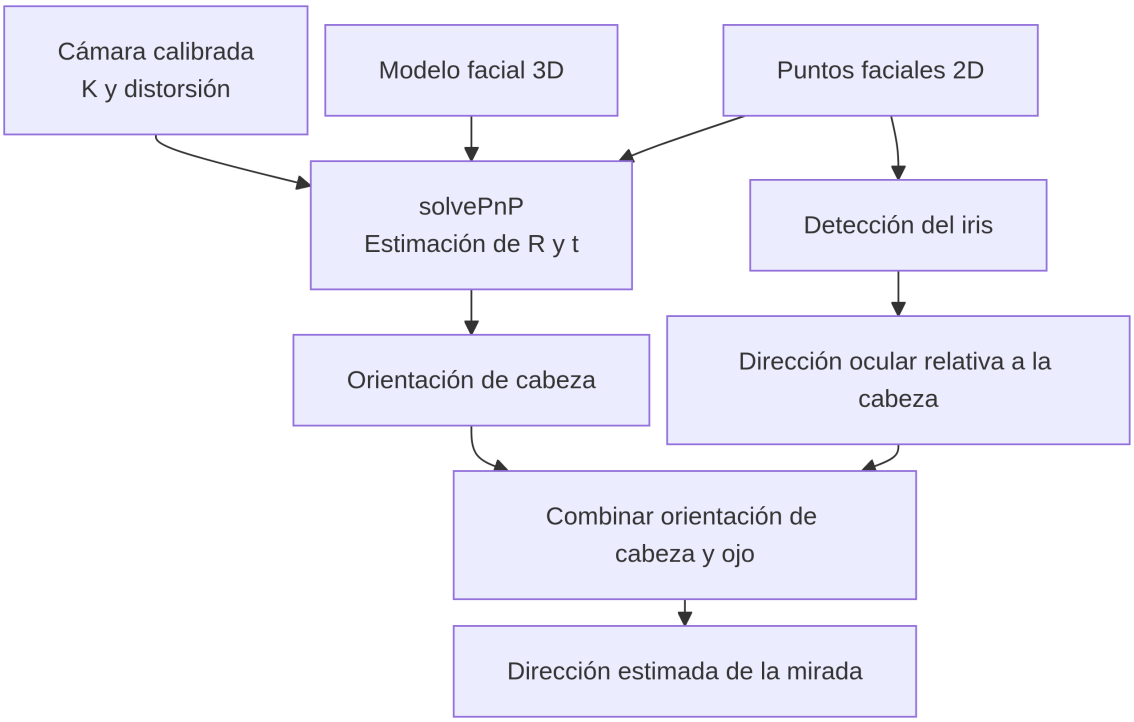


Figura 3.7: Proceso para estimar la orientación de la cabeza y la dirección de la mirada.

Sin embargo, la orientación de la cabeza no tiene por qué coincidir con la orientación de la mirada, ya que los ojos tienen movilidad independiente. Para estimar la mirada con mayor precisión es necesario detectar los iris y estimar la orientación de los ojos respecto a la cabeza.

Finalmente, combinando la rotación de la cabeza con la dirección ocular estimada, se podría obtener una aproximación de la dirección tridimensional de la mirada respecto a la cámara.

4 Flujo óptico y seguimiento

4.1. Ejercicio 4.1

4.1.1. Lucas-kanade optical flow

El algoritmo de Lucas-Kanade permite estimar el movimiento aparente de determinados puntos de interés entre fotogramas consecutivos de un vídeo. Este enfoque se conoce como flujo óptico siperso, ya que únicamente realiza el seguimiento de un conjunto seleccionado de puntos, en lugar de calcular el desplazamiento de todos los puntos de la imagen.

Para se seleccionar el conjunto de puntos deseguimiento se utiliza el detector de esquinas **Shi-Tomasi**. Este algoritmo analiza los gradientes de intensidad en regiones locales de la imagen para identificar puntos con variaciones altas en las dos direcciones independientes.

En la Figura 4.1, se muestran dos puntos seleccionados y sus respectivas ventanas de respuesta con las puntuaciones obtenidas con `cv.cornerMinEigenVal()`, que calcula el menor valor propio de la matriz de gradientes en cada posición de la imagen. Los puntos seleccionados presentan puntuaciones de 0.0678 y 0.0346, respectivamente, ambos corresponden a máximos locales y constituyen candidatos adecuados para el seguimiento óptico.

> **Nota**
>
> La matriz de gradientes, resume cómo varía la intensidad en diferentes direcciones dentro de una región.
>
> $$M = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$
>
> A partir de esta matriz se obtienen dos valores propios, $(\lambda_1 y \lambda_2)$, que describen la magnitud de las variaciones de intensidad en las dos direcciones principales de la región. Shi-Tomasi se queda con el menor valor propio.
>
> $$R = \text{mín}(\lambda_1, \lambda_2)$$
>
> Porque para considerar un región buena como esquina se busca que las dos direcciones presenten variaciones significativas, no sólo una.

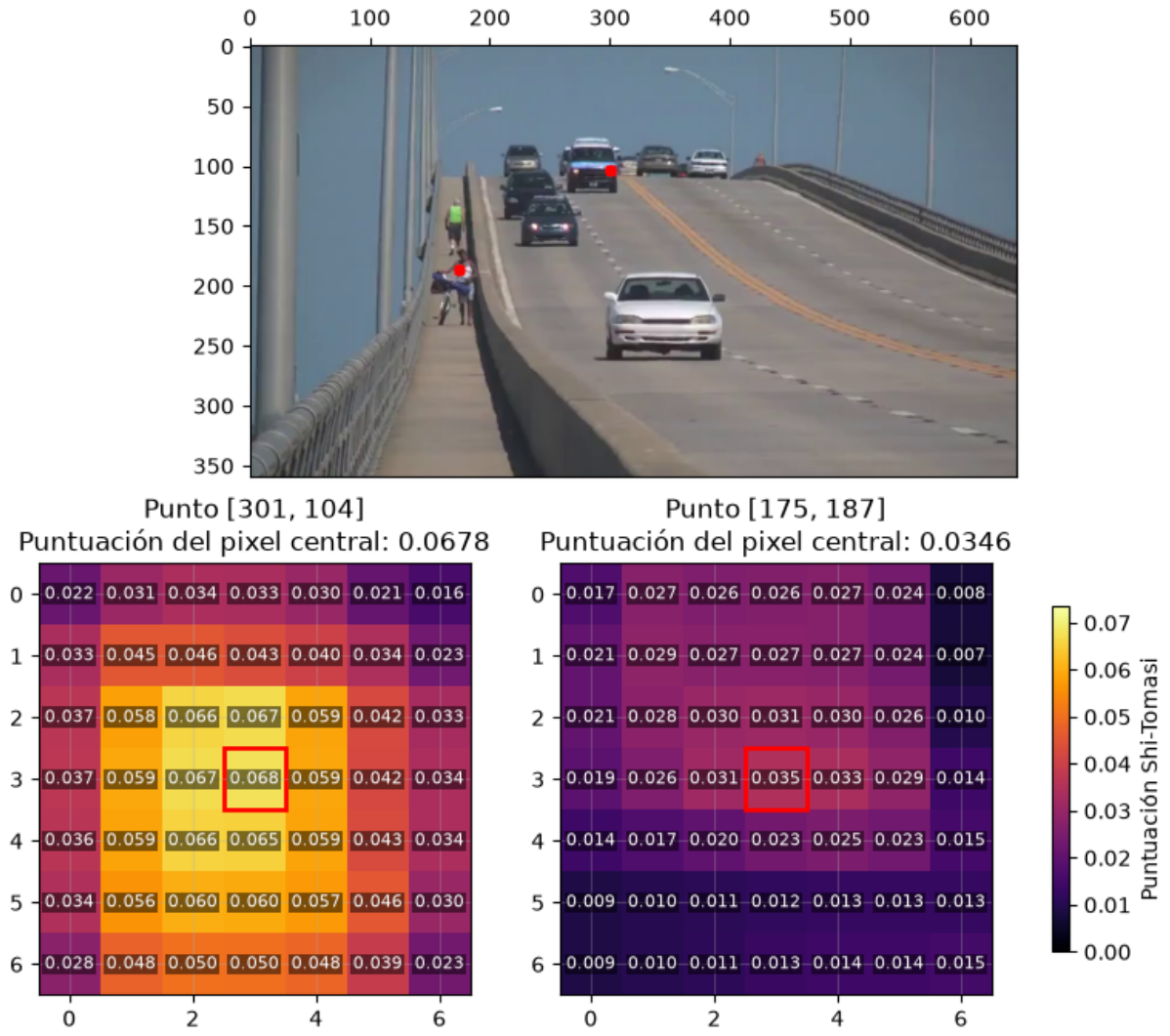


Figura 4.1: Detección de puntos de interés mediante Shi-Tomasi y respuesta local de esquina

Una vez detectados los puntos de interés, el algoritmo Luca-Kanade estima sus nuevas posiciones en el siguiente fotograma mediante `cv.calcOpticalFlowPyrLK()`.

Este método se basa en dos hipótesis principales: que la intensidad de un mismo punto permanece contante entre fotogramas consecutivos y que los píxeles de una pequeña región presentan un movimiento similar. A partir de estas hipótesis, Lucas-Kanade relaciona los gradientes espaciales y temporales de intensidad mediante la ecuación del flujo óptico:

$$I_x u + I_y v + I_t = 0$$

donde I_x e I_y representan los gradientes espaciales, I_t el gradiente temporal y (u, v) las componentes del movimiento. Como esta ecuación contiene dos incógnitas, el algoritmo considera una ventana de píxeles vecinos y supone que comparten un movimiento similar. De esta manera, obtiene un sistema de ecuaciones que resuelve mediante mínimos cuadrados para estimar el desplazamiento del punto [10].

En la Figura 4.2 se muestra el desplazamiento estimado de varios puntos característicos de un vehículo entre dos fotogramas consecutivos. Los puntos verdes representan las posiciones anteriores y los azules las posiciones estimadas en el nuevo fotograma.



Figura 4.2: Desplazamiento de puntos característicos entre dos fotogramas mediante Lucas-Kanade

El seguimiento sucesivo de estos puntos permite reconstruir sus trayectoria a lo largo del vídeo, como se observa en la Figura 4.3.

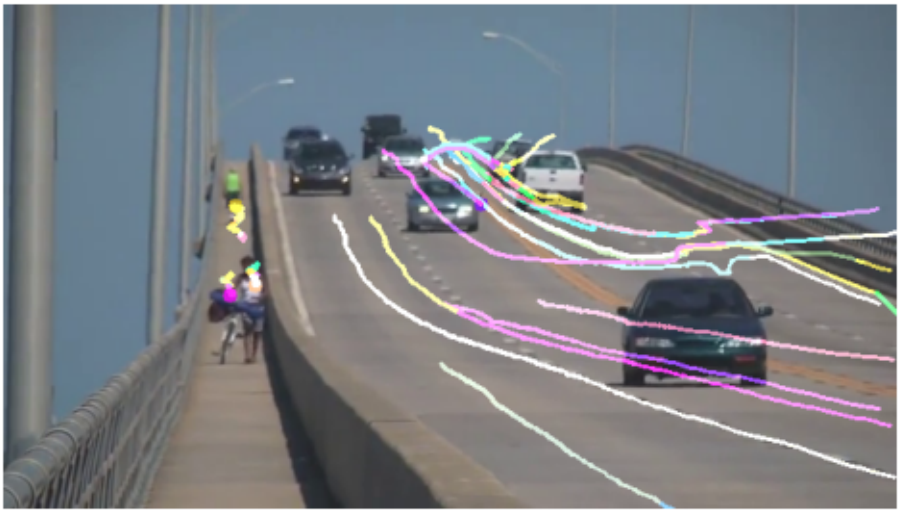


Figura 4.3: Trayectorias de puntos característicos durante el seguimiento óptico

Advertencia

Una de las principales limitaciones de Lucas-Kanade es que en la regiones uniformes o suavizadas de la imagen, las variaciones espaciales de intensidad son pequeñas y, por tanto, no existe suficiente información para determinar de forma fiable el movimiento. Cuando una región presenta una intensidad prácticamente uniforme:

$$I_x \approx 0, \qquad I_y \approx 0$$

Por tanto:

$$M \approx \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Y los valores propios serán aproximadamente:

$$\lambda_1 \approx 0, \qquad \lambda_2 \approx 0$$

Esto implica que no hay suficiente información para determinar de manera fiable las dos componentes del movimiento.

4.1.2. Dense optical flow

El programa **optical_flow1.py** utiliza el algoritmo de Gunnar Farneback para calcular el flujo óptico denso. A diferencia del método de Lucas-Kanade, este enfoque calcula un vector de desplazamiento para cada posición de la imagen.

El algoritmo Farnebäck estima el vector de desplazamiento (u, v) que satisface aproximadamente la relación:

$$I_1(x + u,\ y + v) \approx I_0(x, y)$$

donde I_0 e I_1 representan dos fotogramas consecutivos de una secuencia de vídeo.

El movimiento aparerente entre dos fotogramas se calcula mediante aproximaciones polinómicas de segundo grado. Estas aproximaciones permiten describir las variaciones de intensidad de una región y analizar cómo cambian entre dos fotogramas para determinar su desplazamiento [11].

En la Figura 4.4 se muestran dos fotogramas consecutivos de la secuencia analizada, en ella se observa el desplazamiento de varios peatones.

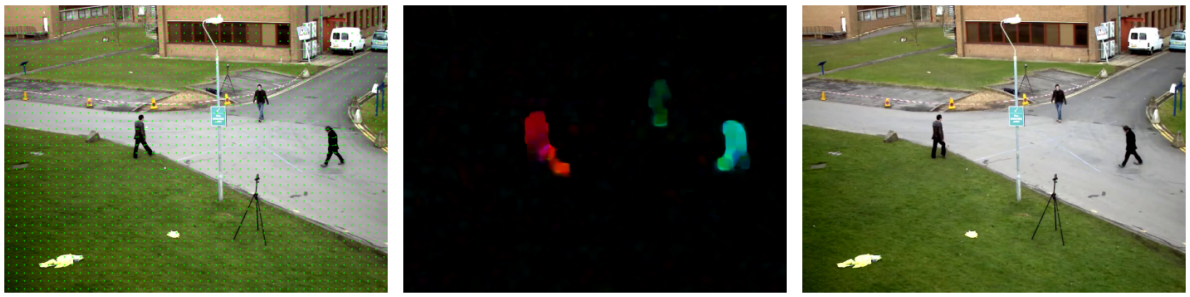


Figura 4.4: Fotogramas consecutivos utilizados para estimar el flujo óptico denso

Mediante la función `cv.calcOpticalFlowFarneback()` se obtiene los vectores de desplazamiento (f_x, f_y) de cada píxel entre los dos fotogramas.

En la Figura 4.5 se muestran tres formas de representar el flujo óptico estimado: - **(a) Representación vectorial:** se visualizan los deplazamientos mediante segmentos dibujados sobre una malla representada sobre la imagen. Su orientación indica la dirección del movimiento y su longitud la magnitud del desplazamiento. - **(b) Representación HSV:** se codifica el flujo óptico mediante colores, el tono representa la dirección del movimiento y el brillo su magnitud normalizada, las regiones con mayor desplazamiento aparecen más brillantes. - **(c) Deformación**

de la imagen: se utiliza el campo de de flujo optico para remuestrear la imagen aplicando un mapeo inverso, para visualizar el efecto estimado del desplazamiento en la imagen.



(a) Vectores de desplazamiento muestreados regularmente

(b) Representación HSV de la dirección y magnitud del movimiento

(c) Deformación de la imagen mediante mapeo inverso a partir del campo de flujo óptico

Figura 4.5: Visualización del flujo óptico denso estimado mediante Farnebäck

Otra forma de visualizar el flujo óptico consiste en construir una imagen RGB, asignando la imagen transformada mediante el campo de flujo al canal rojo y la imagen de referencia a los canales verde y azul.

Cuando ambas imágenes coinciden, el resultado aparece en escala de grises, mientras que las diferencias se manifiestan mediante tonalidades rojizas o cian, permitiendo identificar posibles errores de registro.

En la Figura 4.6 se observa cómo la compensación del movimiento mediante Farnebäck reduce las diferencias entre los fotogramas.



(a) Imagen sin registrar

(b) Secuencia registrada

(c) Objetivo

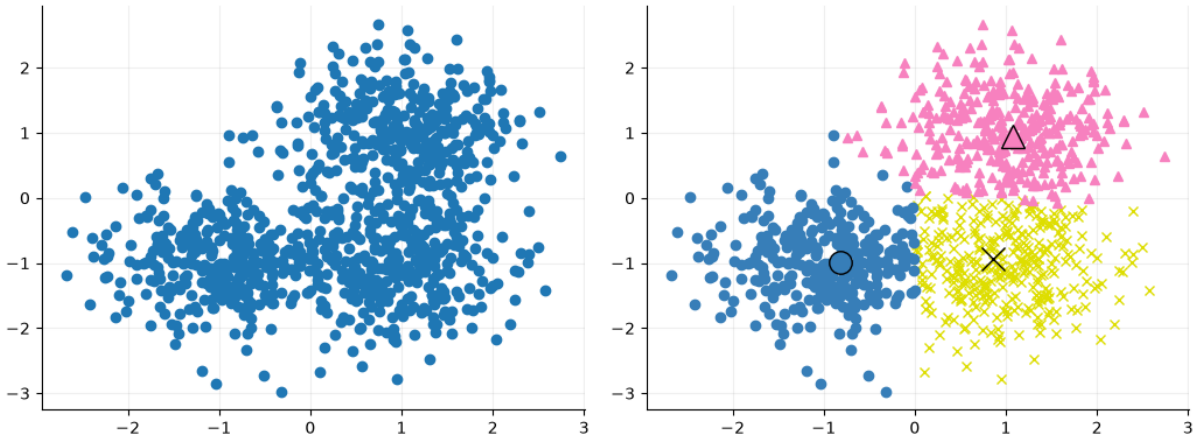
Figura 4.6: Visualización flujo óptico

4.2. Ejercicio 4.2

El algoritmo de **Mean Shift**, situa una ventana alrededr de un punto incial, calcula la media de los puntos contenidos en ella y desplaza la ventana a esa posición, repitiendo el proceso hasta que el desplazamiento es prácticamente nulos permitiendonos localizar zonas de alta densidad como se muestra en la Figura 4.7.

¿Cómo podemos aplicar Mean Shift para realizar el seguimiento de una mano mediante una webcam?

Para realizar el seguimiento de una mano se parte de una imagen de referencia, captura por la webcam, de la que se selecciona una región de interés (ROI) que contiene la mano.

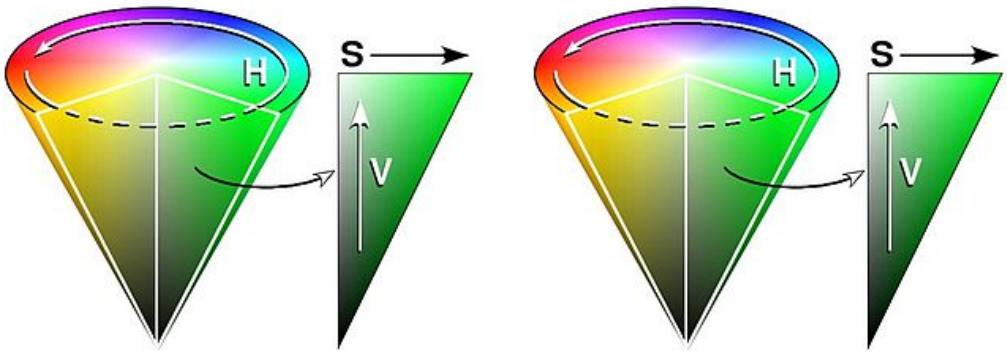


(a) Distribución inicial de los puntos

(b) Agrupaciones y centros detectados mediante Mean Shift

Figura 4.7: Identificación de regiones de alta densidad mediante Mean Shift

Esta región se convierte al espacio de color HSV y se aplica una máscara para seleccionar los píxeles cuyos valores se encuentran dentro de un intervalo determinado. A continuación, se calcula el histograma del canal de tono (H) utilizando los píxeles seleccionados con la máscara y se normalizan los valores entre 0 y 255.



En la Figura 4.8 se muestra la región de interés, la máscara obtenida y el histograma normalizado que caracteriza los tonos de la mano.

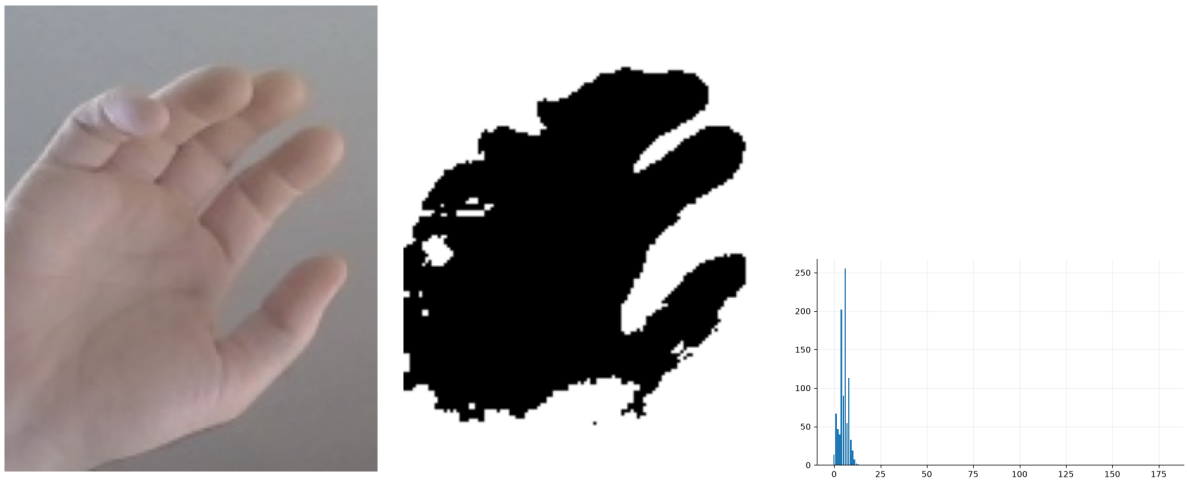
Con el histograma se calcula la retroproyección, utilizando `cv.calcBackProject()`, funciona asignado a cada píxel de la imagen un valor según la correspondencia de su tono, con los tonos normalizados del histograma. De esta forma, se genera una imagen en escala de grises donde los tonos más similares a los de la mano se presentan más oscuros.

En la Figura 4.9a se muestra la retroproyección obtenida con la imagen de referencia. En la Figura 4.9b se representan las agrupaciones detectadas al aplicar MeanShift sobre las coordenadas de los píxeles seleleccionados, podemos observar que, además de la mano aparecen otras regiones similares, especialmente alrededor de la cara.

Para realizar el seguimiento en tiempo real utiliza el comando `uv run miia-pec1 meanshift`.

Algunos consideraciones a tener en cuenta:

- Necesita una ventana inicial para realizar el seguimiento.
- Para que el seguimiento sea correcto es necesario que las condiciones de iluminación sean similares a los de la captura de referencia.

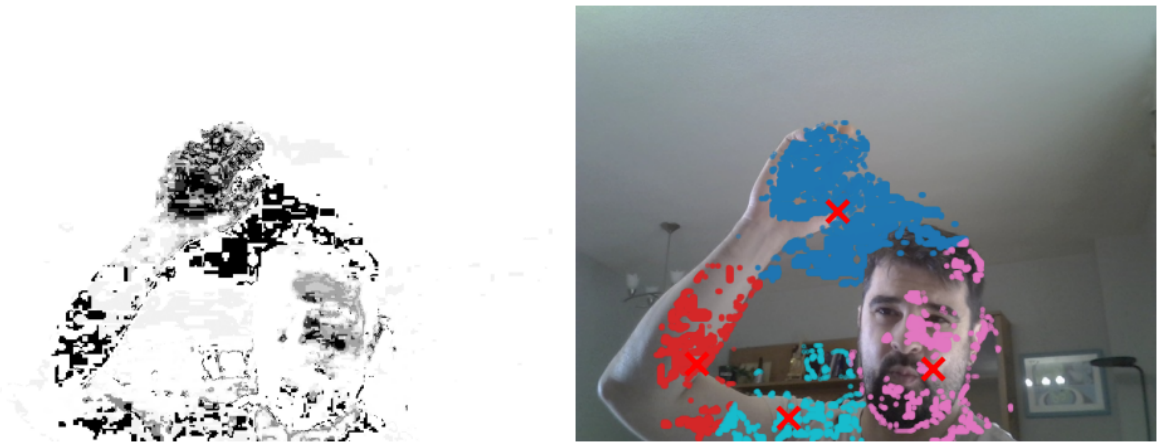


(a) Región de interés (ROI) de la mano

(b) Máscara binaria obtenioda mediante umbrales HSV

(c) Histograma normalizado del canal H

Figura 4.8: Obtención del modelo de color de la mano mediante segmentación HSV e histograma normalizado



(a) Retroproyección del histograma de color

(b) Agrupaciones espaciales identificadas mediante MeanShift

Figura 4.9: Identificación de regiones con colores similares a los de la mano mediante retroproyección y agrupamiento MeanShift

- El tamaño de la ventana es fijo, no se adapta aunque cambie la escala de la mano.
- Se pierde el foco al salir de plano, se realiza un movimiento brusco hacia la ventana inicial o zonas de tonalidades similares a la mano como la cara.

```
# Este comando realiza el seguimiento en la webcam de un objeto usando
 ↪  meanshift
# Recibe una imagen de referencia, el área de interés y los valores mínimo y
 ↪  máximo con los que generar la máscara para el seguimiento.
!uv run miia-pec1 meanshift \
    --track-window 210,110,120,150 \
    --mask-min 0,35,70 \
    --mask-max 20,130,255 \
    --image-ref {OUT_DIR / 'seguimiento.jpg'} \
    --device 2 \
    --outfile {OUT_DIR / 'ejemplo.mp4'}
```

5 Segmentación sin conocimiento del dominio

5.1. Umbralización

5.1.1. Umbralización global

La umbralización global utiliza un único umbral para segmentar toda la imagen.

$$dst(x,y) = \begin{cases} valor_x, & \text{si } src(x,y) > thresh \\ valor_y, & \text{si } src(x,y) \leq thresh \end{cases}$$

Donde:

- $valor_x$: valor asignado cuando se supera el umbral.
- $valor_y$: valor asignado cuando no se supera el umbral.

Los principales tipos de umbralización son:

- **Binaria:** Asigna el valor `maxValue` si el valor del píxel el mayor que el umbral y 0 en caso contrario.
- **Binaria inversa:** Invierte los valores asignados por la umbralización binaria.
- **Truncamiento:** Asigna el valor umbral si el el valor del píxel es mayor que el umbral, de lo contrario asigna el valor original del píxel.
- **Puesta a cero:** Conversa los valores mayores que el umbral y el resto las sustituye por 0.
- **Puesta a cero invertido:** Conserva los valores menores o iguales al umbral y el resto los sustituye por 0.

Los histogramas de la Figura 5.1 permiten analizar cómo se modifica la distribución de intensidades con cada método. Mientras las umbralizaciones binarias concentran los píxeles en dos valores, los métodos de truncamiento y puesta a cero conservan parte de las intensidades originales.

En la Figura 5.2 se puede observar que los resultados son dependientes del contraste y la iluminación de cada imagen.

5.1.2. Umbralización local

La umbralización local, calcula un umbral $T(x,y)$ para cada píxel a partir de una ventana centrada en él.

Dentro de las ventana OpenCV utiliza dos métodos para calcular el umbral:

- **Media:** El umbral es la media del área vecina menos una constante C.
- **Gaussiana:** El umbral es una media ponderada gaussiana del área vecina menos una constante C.

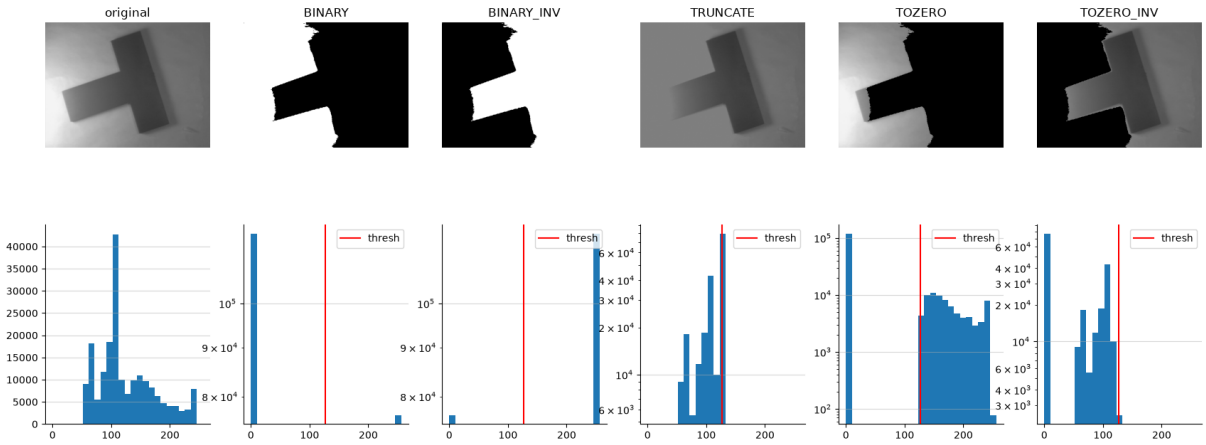


Figura 5.1: Distribución de intensidades tras aplicar los distintos métodos de umbralización global

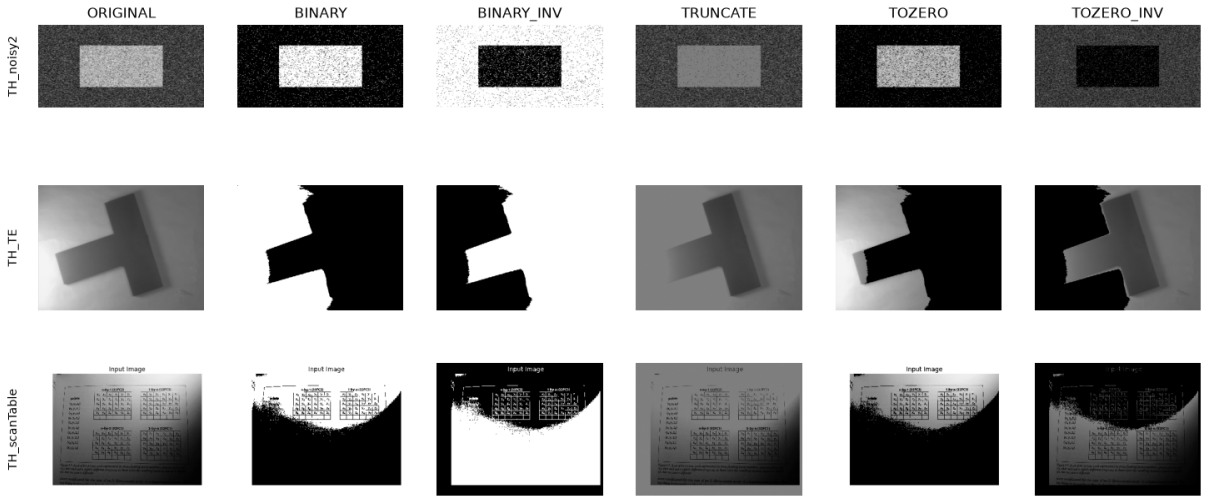


Figura 5.2: Comparación de los métodos de umbralización global (umbral = 127)

En la Figura 5.3 se muestra como la umbralización local, permite recuperar una mayor cantidad de detalles que la umbralización global sobre imágenes que presentan variaciones en la iluminación, ya permite ajustar el umbral de cada zona, las zonas iluminadas tendrán un umbral alto mientras que las oscuras presentarán uno más bajo.

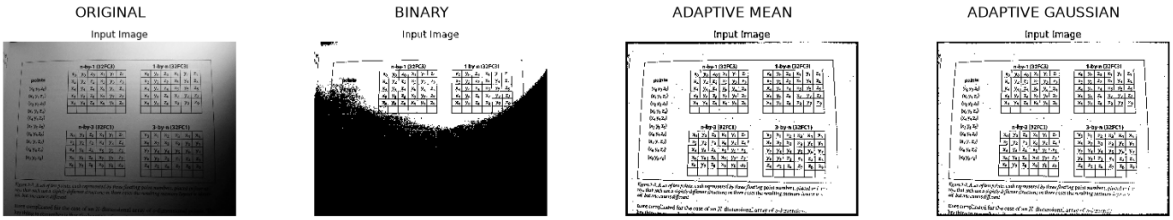


Figura 5.3: Umbralización global vs umbralización local

En la Figura 5.4 se observa como al aumentar el valor de C se reduce la presencia de píxeles negros aislados en el fondo, mientras que el valor de `blockSize` determina el tamaño de la la extensión que se utiliza para calcular el umbral.

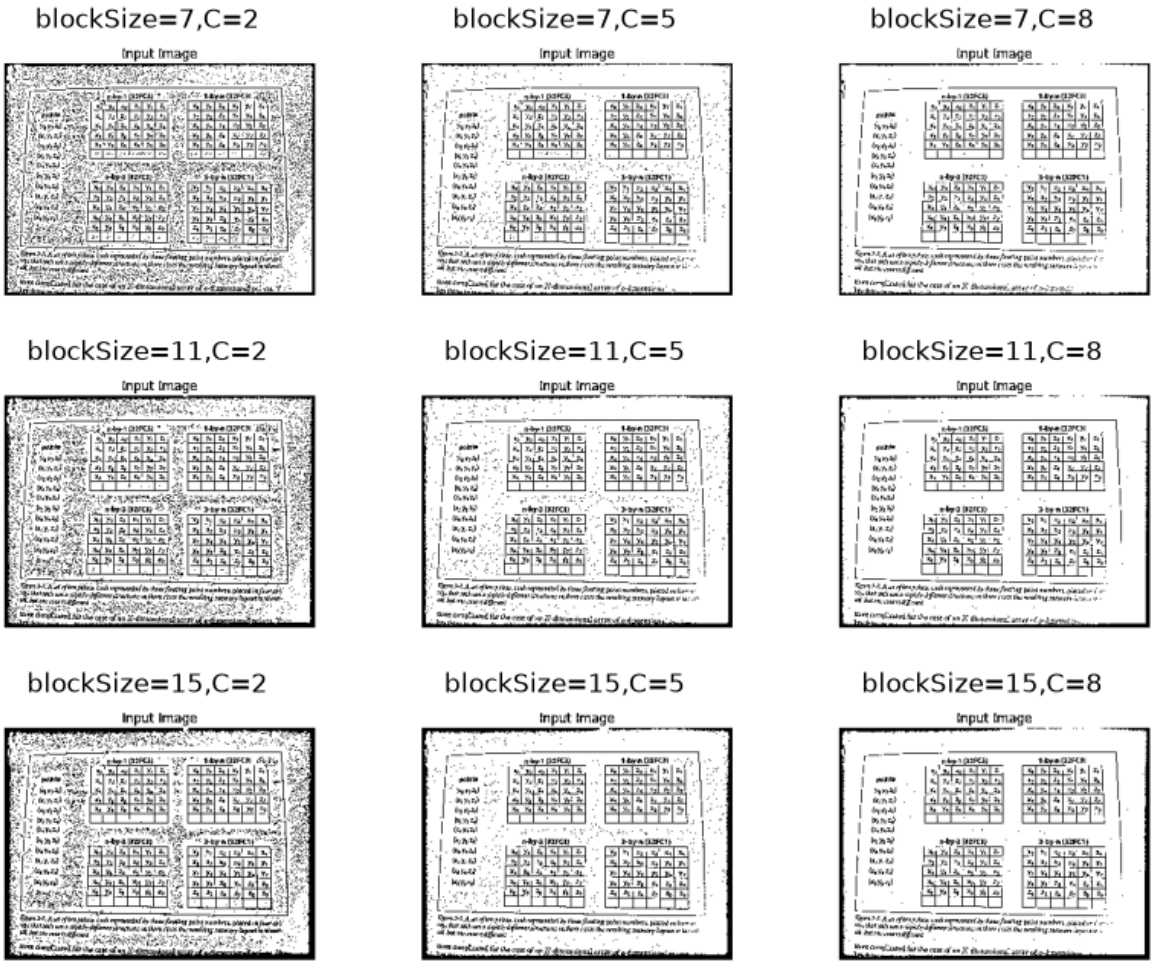


Figura 5.4: Ajuste de parámetros para la umbralización local

Se analiza si es posible segmentar completamente el réctangulo de la imagen TH_noisy2.png o la pieza en forma de T de TH_TE.gif utilizando únicamente umbralazación local.

Para analizar el efecto del tamaño de la ventana, sobre el resultado, se selecciona un recorte que

contiene parte del rectángulo y del fondo.

En la Figura 5.5, se observa como una ventana más pequeña, produce una segementación más fragmentada, dado que el umbral se calcula a partir de una región reducida de píxeles vecinos que tienen intensidades similares al píxel central de la ventana. Al aumentar el tamaño de la ventana está incorpora píxeles de las dos regiones del recorte lo que favorece su diferenciación asemejando su comportamiento a una umbralización global.



(a) Recorte de una región (b) Umbralización con una ventana pequeña (blockSize=15) (c) Umbralización con una ventana grande (blockSize=81)

Figura 5.5: Análisis de la umbralización local en regiones homogéneas

Como se muestra en las figuras Figura 5.6 y Figura 5.7, es posible obtener una segmentación bastante completa utilizando tamaños de ventana grandes, para que las ventanas incluyan píxeles del objeto que estamos buscando segmentar y del fondo.

En el caso del rectángulo se obtiene un buen resultado con `blockSize=511` y $C = 10$, mientras que para la pieza en forma de T el mejor resultado se obtiene con `blockSize=501` y $C = 20$.

Sin embargo, ninguna de las configuraciones consigue una segmentación completa y sin errores, habría que considerar otras técnicas.

5.2. K-means clustering

La imagen brain1.png corresponde a una resonancia magnética cerebral en escala de grises.

La Figura 5.8 muestra la imagen original con su histograma de intensidades, se observa una concentración elevada en las intensidades bajas, asociadas a las regiones oscuras y al fondo de la imagen.

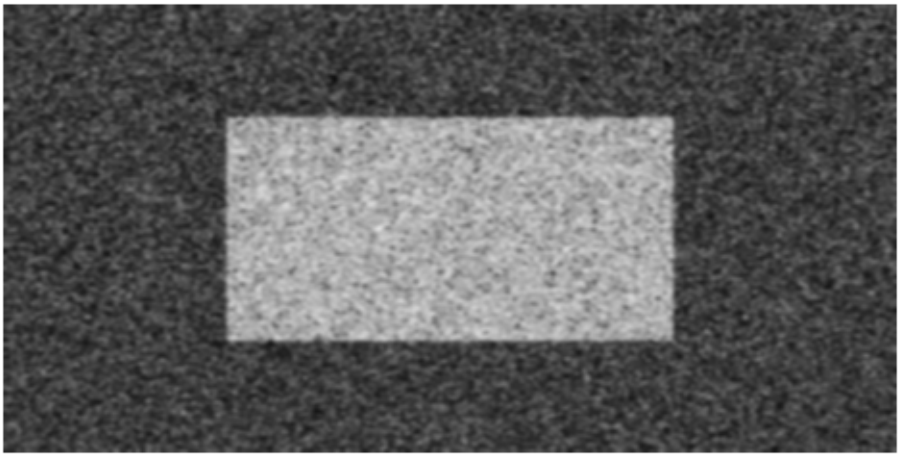
se muestra el histograma de la imagen para ver como se reparte la intensidad de grises en la imagen.

La Figura 5.9 muestra los resultados de aplicar el K-means con diferente número de clústeres. Para cada configuración se muestra la imagen segmentada junto con el histograma de intensidades original, coloreado según los grupos obtenidos, las líneas verticales naranjas indican la posición de los centroides calculados por el algoritmo.

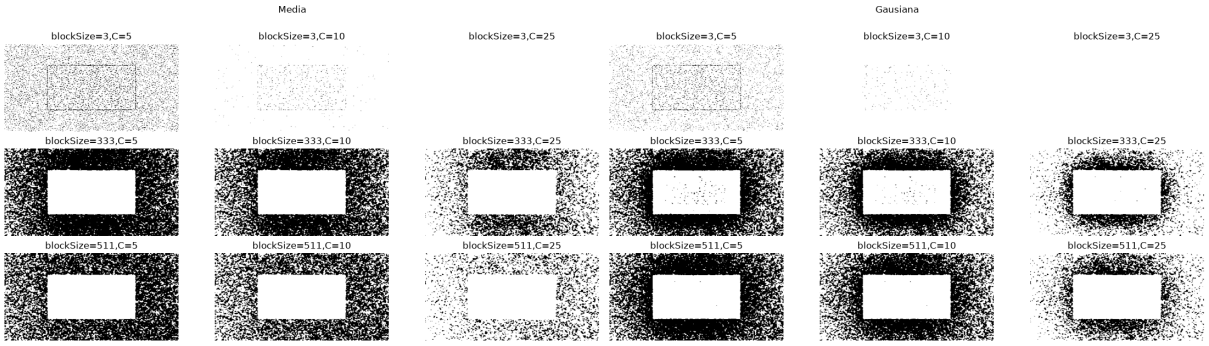
Para valores reducidos de k, como k=2 y k=3, la segmentación permite diferenciar regiones más oscuras de las estructuras del cerebro, pero se pierde el detalle anatómico.

Al aumentar el número de clusters k=5 y k=7, presenta una mayor variedad de intensidades intermedias y altas. Esto permite visualizar más estructuras del tejido cerebral.

Para k=12 y k=15 se incorporan nuevos centroides que mejoran la representación de las transiciones entre niveles de gris permitiendo distinguir más detalles de la imagen.



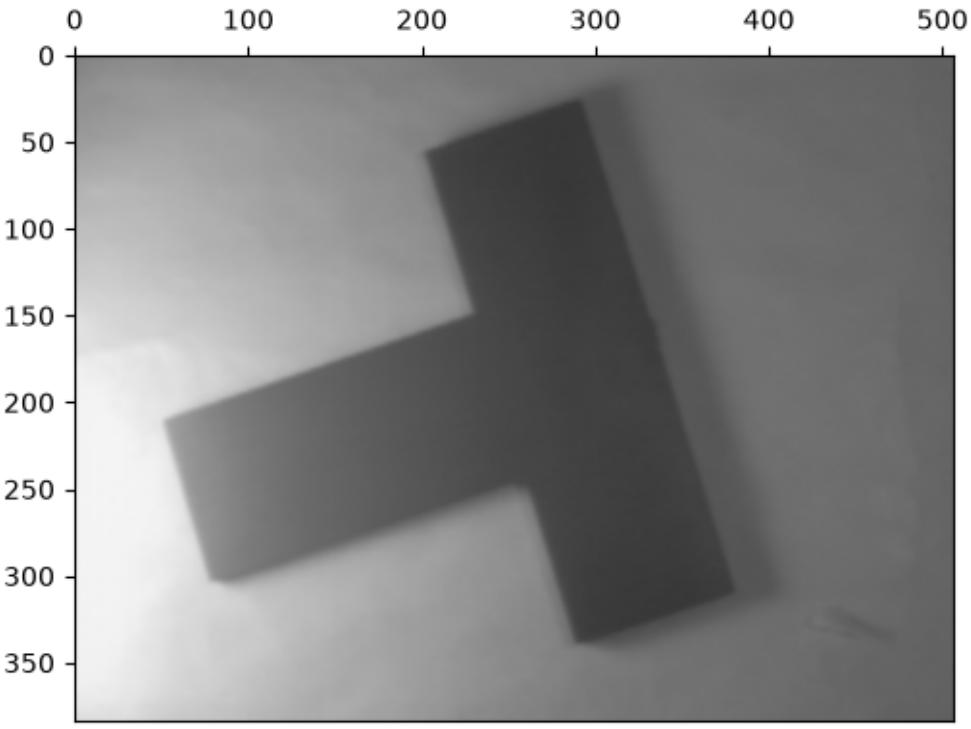
(a) Imagen original



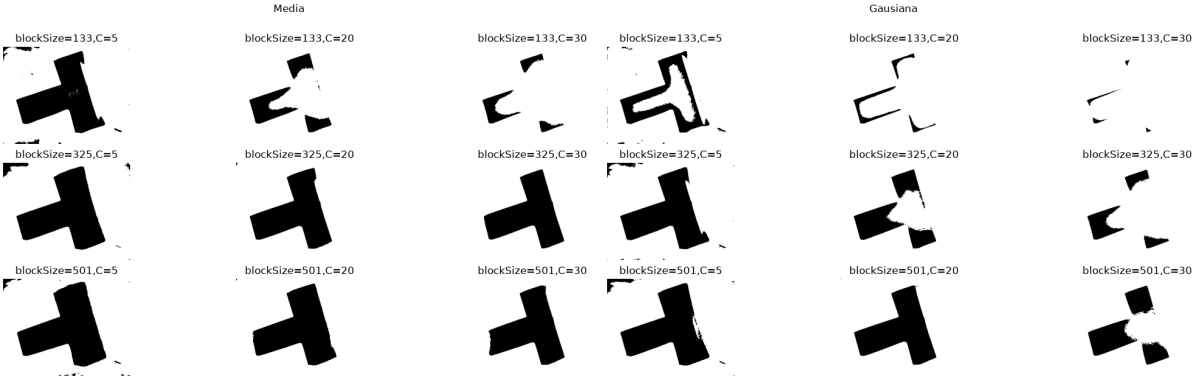
(b) Aplicación de umbralización local, usando la media

(c) Aplicación de umbralización local, usando la gausiana

Figura 5.6: Ajuste de parámetros para la umbralización local



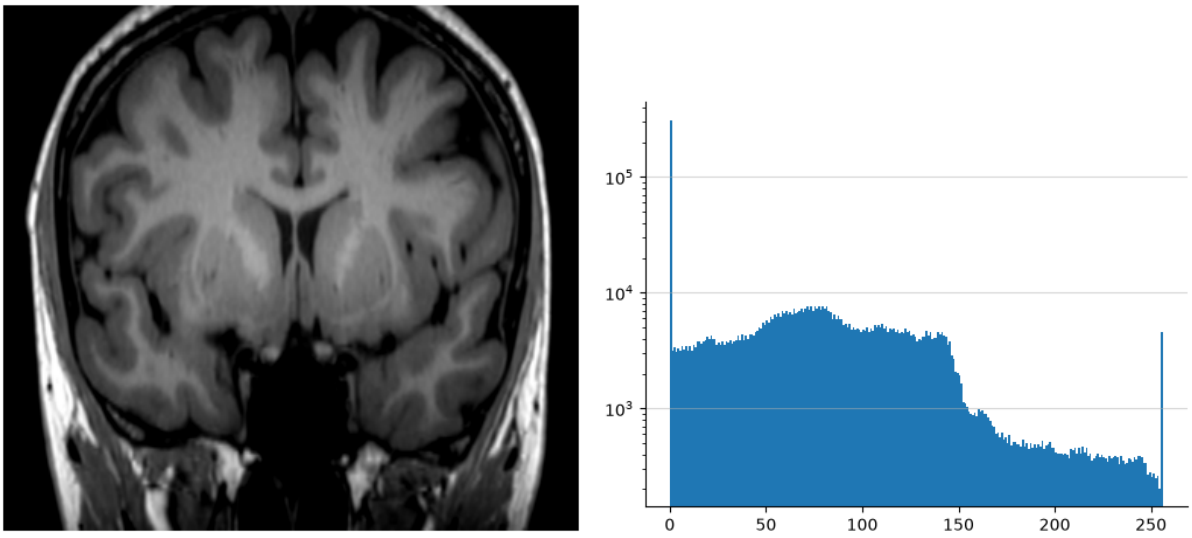
(a) Imagen original



(b) Aplicación de umbralización local, usando la media

(c) Aplicación de umbralización local, usando la gausiana

Figura 5.7: Ajuste de parámetros para la umbralización local



(a) Imagen original (b) Histograma de intensidades de la imagen original

Figura 5.8: Resonancia magnética cerebral

Los resultados muestran que aumentar el número de clústeres mejora la fidelidad de la imagen segmentada pero aumenta la complejidad de la imagen.

5.3. Segmetación con MeanShift

El algoritmo Mean Shift que nos permite suavizar una imagen conservando los bordes. OpenCV proporciona una implementación de este algoritmo, funciona desplazando de forma iterativa una ventana de búsqueda hacia regiones de mayor densidad en el espacio conjunto de espacio y color. Para ello, se consideran los píxeles que cumplen las siguientes condiciones:

$$\left\{(x,y): \begin{array}{r} X-\texttt{sp} \leq x \leq X+\texttt{sp}, \\ Y-\texttt{sp} \leq y \leq Y+\texttt{sp}, \\ \|(R,G,B)-(r,g,b)\|_2 \leq \texttt{sr} \end{array}\right\}$$

Donde:

- **sp:** representa el radio de la ventana espacial y determina la distancia máxima considerada entre píxeles vecinos.
- **sr:** establece el radio de similitud cromática, limitando la diferencia de color entre los píxeles considerados.
- **(X,Y):** corresponde a la posición central de la ventana de búsqueda.
- **(R,G,B):** representa el color de referencia de la ventana.

Mediante este proceso, los píxeles con características espaciales y cromáticas similares convergen hacia valores representativos próximos, dando lugar a regiones de color más uniforme.

La Figura 5.10 presenta los resultados de aplicar Mean Shift con diferentes valores de los parámetros sp y sr, con el objetivo de analizar su influencia en el suavizado y la conservación de los detalles.

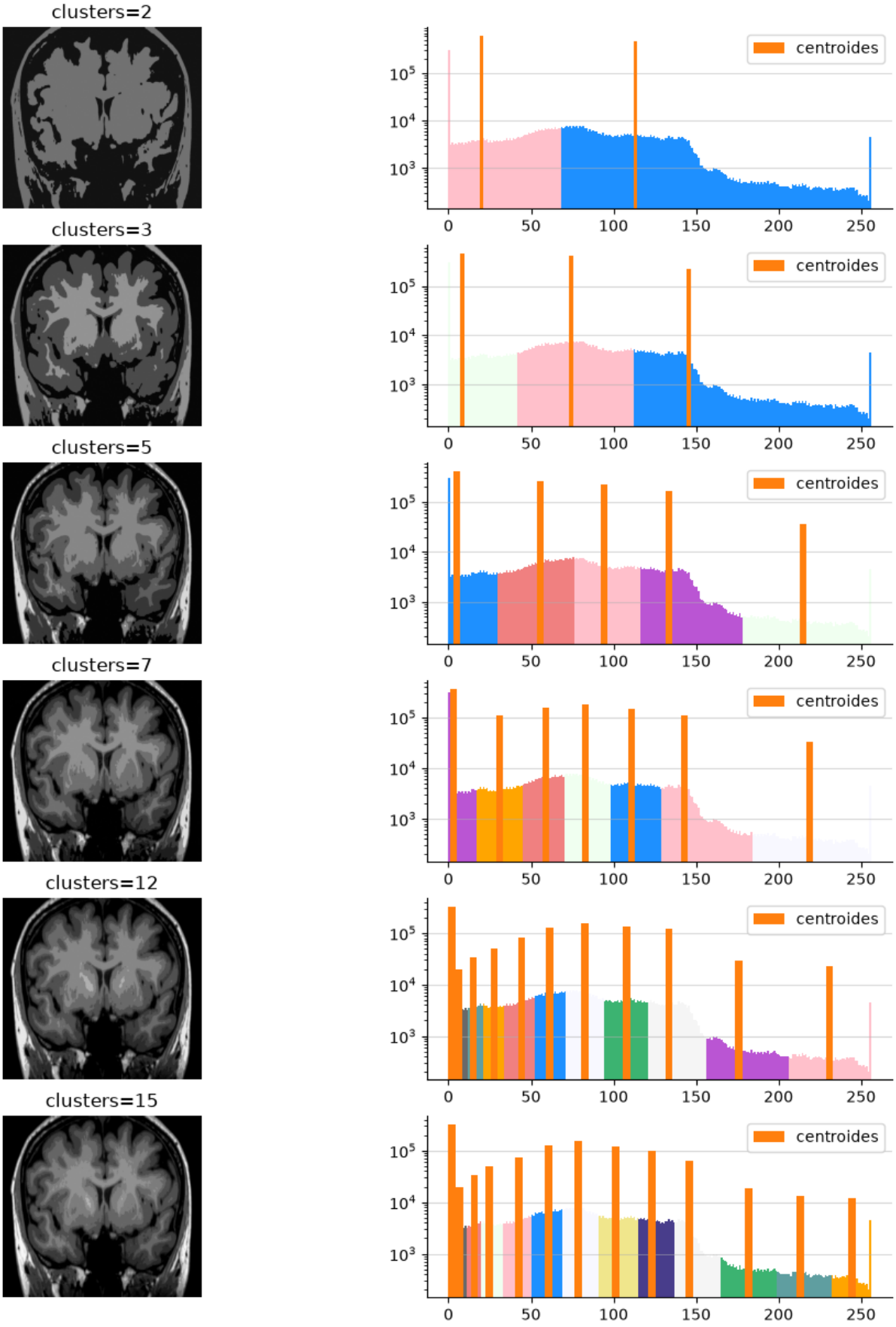


Figura 5.9: Segmentación mediante K-means para distintos números de clústeres

En la Figura 5.10a se aumenta el radio espacial, manteniedo el mismo radio de color. Al aumentar el radio espacial, el algoritmo considera vecinos píxeles más alejados favoreciendo la formación de regiones homogéneas más grandes, se reducen detalles pero las estructuras principales permanecen reconocibles.

En la Figura 5.10b mantiene el radio espacial y se incrementa el radio de color. El aumento del radio de color permite agrupar píxeles con diferencias de color mayores, lo que se traduce en un suavizado más intenso.



(a) Variación del radio espcial



(b) variación del radio de color

Figura 5.10: Efecto de los parámetros en el filtrado Mean Shift

El parámetro maxLevel, establece el nivel máximo de una pirámide guassiana empleada durante el procesamiento en maxLevel+1.

En la Figura 5.12 se comparan los resultados con distintos niveles de maxLevel. Las tres configuraciones mantienen las principales regiones de color y la estructura general de la escena, las diferencia aparecen principalmente en las zonas con variaciones locales de color y en delimitación de algunas regiones.

5.4. Segmentación Watershed

En la figura Figura 5.13 se muestra la segmentación de los lóbulos cerebrales utilizando un umbral de Otsu sobre la imagen suavizada y enmascarada con una elipse que representa la región del cerebro.

En la Figura 5.14a se muestran las transformaciones de fondo seguro, primer plano seguro y región desconocida que se utilizan para construir la máscara de regiones segmentadas, Figura 5.14b, que es utilizada como semilla por el algoritmo de watershed.

En la Figura 5.15 se muestra el resultado de la aplicación del algoritmo watershed para la separación de los lóbulos.

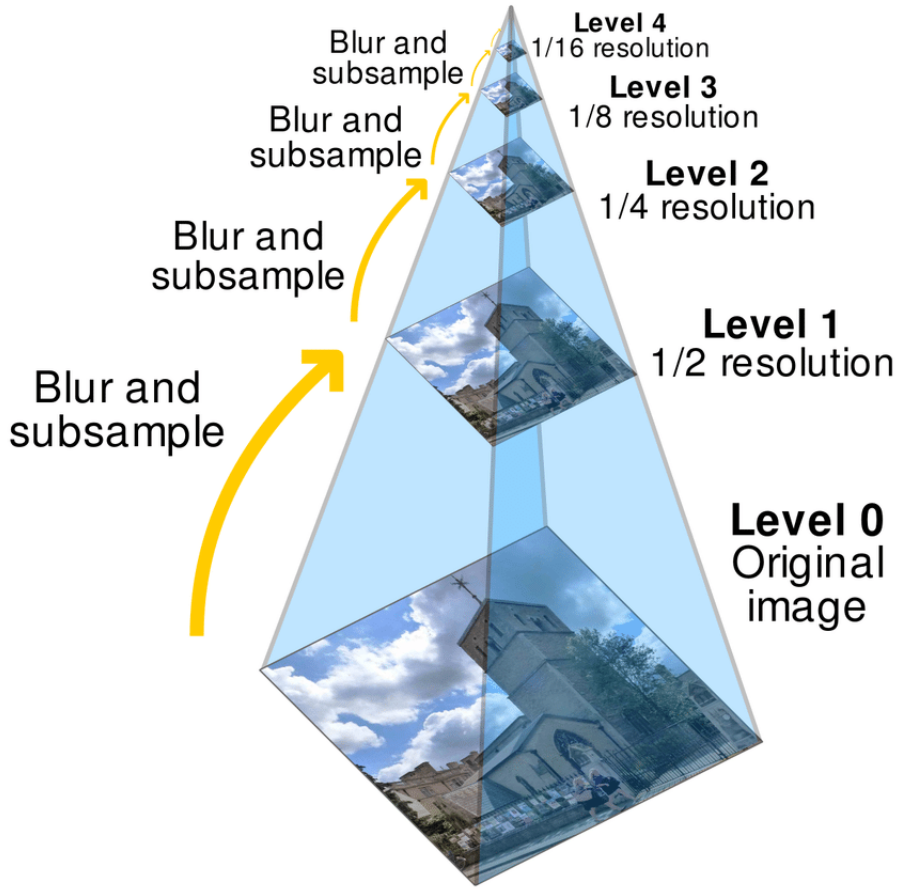


Figura 5.11: Pirámide gaussiana. Fuente: Branch y Stewart (2018) [12].

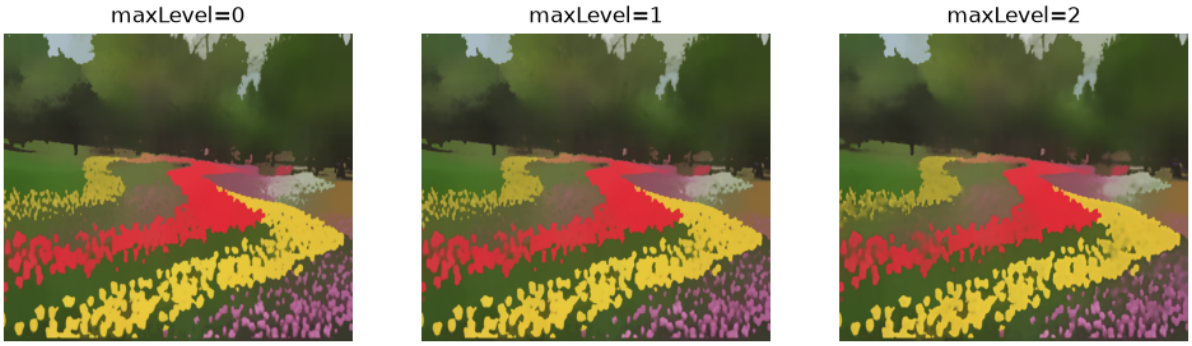


Figura 5.12: Resultado de pirámides gaussianas al aplicar mean shift



Figura 5.13: Segmentación de los lóbulos cerebrales



(a) Fondo seguro, primer plano seguro y región desconocida



(b) Máscara de las regiones segmentadas

Figura 5.14: Transformaciones intermedias

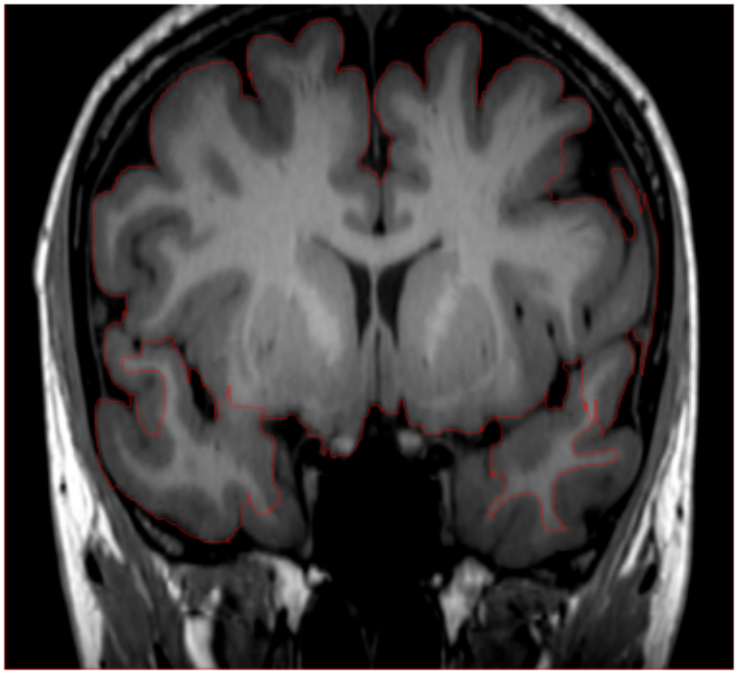


Figura 5.15: Separación de los lóbulos con watershed

Referencias

[1] Posit Software, PBC, *Quarto*. (2026). Disponible en: https://quarto.org

[2] Hugging Face, «Hugging Face: The AI Community Building the Future». https://huggingface.co/.

[3] P. Moracho, «Respuesta a: ¿Cómo usar contours apropiadamente?» Stack Overflow en español, 13 de abril de 2018. Disponible en: https://es.stackoverflow.com/a/155697/510086

[4] OpenCV, «Geometric Transformations of Images». Accedido: 16 de octubre de 2026. [En línea]. Disponible en: https://docs.opencv.org/5.0/py_tutorials/py_imgproc/py_geometric_transformations/py_geometric_transformations.html#rotation

[5] A. Rosebrock, «Rotate Images (Correctly) with OpenCV and Python». Accedido: 15 de octubre de 2026. [En línea]. Disponible en: https://pyimagesearch.com/2017/01/02/rotate-images-correctly-with-opencv-and-python/

[6] Wikipedia contributors, «Matrix (mathematics)». Accedido: 17 de octubre de 2026. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Matrix_(mathematics)

[7] OpenCV, «Geometric Image Transformations». Accedido: 17 de octubre de 2026. [En línea]. Disponible en: https://docs.opencv.org/5.0/main_modules/imgproc_transform.html

[8] Wikipedia contributors, «Nearest-neighbor interpolation». Accedido: 17 de octubre de 2026. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Nearest-neighbor_interpolation

[9] OpenCV, «3D vision functionality: solvePnP». Disponible en: https://docs.opencv.org/5.0/main_modules/d_projection.html#solvepnp

[10] OpenCV, «Optical Flow». OpenCV Documentation. Disponible en: https://docs.opencv.org/doc/doxygen/html/d4/dee/tutorial_optical_flow.html

[11] G. Farnebäck, «Two-Frame Motion Estimation Based on Polynomial Expansion», en *Image Analysis: 13th Scandinavian Conference, SCIA 2003*, J. Bigun y T. Gustavsson, Eds., en Lecture Notes en Computer Science, vol. 2749. Springer, 2003, pp. 363-370. doi: 10.1007/3-540-45103-X_50.

[12] N. Branch y E. C. Stewart, «Applications of Phase-Based Motion Processing», 2018. Disponible en: https://www.researchgate.net/figure/Gaussian-Image-Pyramid_fig7_322310138